Modelos de regresión en Machine Learning

Edison Achalma
Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga

Resumen

Este abstract será actualizado una vez que se complete el contenido final del artículo.

Palabras Claves: keyword1, keyword2

Tabla de contenidos

Introduction **5**

1 La regresión **5**
1.1 Concepto . . . 5
1.2 Tipos de problemas de regresión . . . 5
1.3 Generar datos de ejemplo . . . 6
1.4 Dividir datos . . . 8

2 Mínimos cuadrados ordinarios (OLS) **8**
2.1 Fundamento matemático . . . 8
2.1.1 Especificación del modelo . . . 8
2.1.2 Función objetivo . . . 9
2.1.3 Derivación de la solución . . . 9
2.1.4 Propiedades del estimador OLS . . . 10
2.1.5 Matriz de varianzas-covarianzas . . . 10
2.1.6 Supuestos del modelo clásico de regresión lineal . . . 10
2.1.7 Ventajas del método OLS . . . 11
2.1.8 Limitaciones del método OLS . . . 11
2.1.9 Aplicaciones en economía . . . 12
2.1.10 Consideraciones computacionales . . . 12
2.2 Implementación . . . 12
2.3 Visualización de predicciones . . . 13
2.4 Análisis de residuos . . . 14

Edison Achalma https://orcid.org/0000-0001-6996-3364

El autor no tiene conflictos de interés que revelar. Los roles de autor se clasificaron utilizando la taxonomía de roles de colaborador (CRediT; https://credit.niso.org/) de la siguiente manera: Edison Achalma: conceptualización, metodología, análisis formal, investigación, recursos, curación de datos, redacción, visualización, supervisión, administración del proyecto

La correspondencia relativa a este artículo debe dirigirse a Edison Achalma, Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga, Portal Independencia N° 57, Ayacucho, AYA 5001, Perú, Email: elmer.achalma.09@unsch.edu.pe

3 Regresión Ridge **16**
3.1 Fundamento matemático . . . 16
3.1.1 Especificación del problema . . . 16
3.1.2 Derivación de la solución . . . 17
3.1.3 Interpretación geométrica . . . 17
3.1.4 Efecto del parámetro de regularización α . . . 17
3.1.5 Propiedades del estimador Ridge . . . 18
3.1.6 Trade-off sesgo-varianza . . . 18
3.1.7 Interpretación Bayesiana . . . 18
3.1.8 Ventajas de la regresión Ridge . . . 19
3.1.9 Limitaciones de la regresión Ridge . . . 19
3.1.10 Estandarización de variables . . . 19
3.1.11 Aplicaciones en economía . . . 20
3.1.12 Selección del parámetro α mediante validación cruzada . . . 20
3.2 Implementación . . . 20
3.3 Visualización del efecto de regularización . . . 21

4 Regresión Lasso **23**
4.1 Fundamento matemático . . . 23
4.1.1 Especificación del problema . . . 23
4.1.2 Diferencia fundamental con Ridge . . . 23
4.1.3 Solución del problema Lasso . . . 24
4.1.4 Interpretación geométrica . . . 24
4.1.5 Propiedades de soft-thresholding . . . 24
4.1.6 Efecto del parámetro α . . . 25
4.1.7 Propiedades del estimador Lasso . . . 25
4.1.8 Comparación Ridge vs Lasso . . . 25
4.1.9 Ventajas de la regresión Lasso . . . 26
4.1.10 Limitaciones de la regresión Lasso . . . 26
4.1.11 Interpretación Bayesiana . . . 26
4.1.12 Estandarización de variables . . . 27
4.1.13 Aplicaciones en economía . . . 27
4.1.14 Selección del parámetro α . . . 27
4.1.15 Lasso adaptativo . . . 28
4.2 Implementación . . . 28
4.3 Comparación Ridge vs Lasso . . . 29

5 Elastic Net **31**
5.1 Fundamento matemático . . . 31
5.1.1 Especificación del problema . . . 31
5.1.2 Forma alternativa de la penalización . . . 31
5.1.3 Casos especiales . . . 31
5.1.4 Motivación: limitaciones de Lasso . . . 32
5.1.5 Propiedades del estimador Elastic Net . . . 32
5.1.6 Algoritmo de optimización . . . 32
5.1.7 Interpretación geométrica . . . 33
5.1.8 Ventajas de Elastic Net . . . 33
5.1.9 Limitaciones de Elastic Net . . . 33

5.1.10 Selección de hiperparámetros . . . 34
5.1.11 Comparación de los tres métodos . . . 34
5.1.12 Aplicaciones en economía . . . 34
5.1.13 Recomendaciones prácticas . . . 35
5.1.14 Extensiones y variantes . . . 35
5.2 Implementación . . . 35

6 Regresión polinomial **36**
6.1 Fundamento matemático . . . 36
6.1.1 Especificación del modelo univariado . . . 36
6.1.2 Transformación de características . . . 36
6.1.3 Regresión polinomial multivariada . . . 37
6.1.4 Número de términos en el modelo . . . 37
6.1.5 Propiedades matemáticas . . . 37
6.1.6 Interpretación de coeficientes . . . 38
6.1.7 Problemas de multicolinealidad . . . 38
6.1.8 Sesgo y varianza con el grado del polinomio . . . 39
6.1.9 Selección del grado óptimo . . . 39
6.1.10 Ventajas de la regresión polinomial . . . 39
6.1.11 Limitaciones de la regresión polinomial . . . 40
6.1.12 Aplicaciones en economía . . . 40
6.1.13 Alternativas modernas . . . 41
6.1.14 Recomendaciones prácticas . . . 41
6.2 Implementación . . . 41

7 K-Nearest Neighbors Regressor **44**
7.1 Fundamento matemático . . . 44
7.1.1 Definición formal . . . 44
7.1.2 Métricas de distancia . . . 44
7.1.3 Algoritmo KNN . . . 45
7.1.4 Variantes ponderadas . . . 46
7.1.5 Propiedades estadísticas . . . 46
7.1.6 Selección del parámetro k . . . 47
7.1.7 Complejidad computacional . . . 47
7.1.8 Sensibilidad a la escala . . . 48
7.1.9 Maldición de la dimensionalidad . . . 48
7.1.10 Ventajas del método KNN . . . 48
7.1.11 Limitaciones del método KNN . . . 49
7.1.12 Aplicaciones en economía . . . 49
7.1.13 Comparación con métodos paramétricos . . . 50
7.1.14 Extensiones y variantes . . . 50
7.1.15 Recomendaciones prácticas . . . 51
7.2 Implementación . . . 51

8 Métricas de evaluación **53**
8.1 Fundamento matemático . . . 53
8.1.1 Notación . . . 53
8.1.2 Mean Squared Error (MSE) . . . 53
8.1.3 Root Mean Squared Error (RMSE) . . . 54

8.1.4 Mean Absolute Error (MAE) . . . 55
8.1.5 Comparación MAE vs RMSE . . . 55
8.1.6 Coeficiente de Determinación (R^2) . . . 56
8.1.7 R^2 Ajustado . . . 57
8.1.8 Mean Absolute Percentage Error (MAPE) . . . 57
8.1.9 Median Absolute Error (MedAE) . . . 58
8.1.10 Elección de métrica según contexto . . . 59
8.1.11 Tabla comparativa de métricas . . . 59
8.1.12 Relaciones matemáticas entre métricas . . . 59
8.1.13 Recomendaciones prácticas . . . 60
8.2 Implementación de métricas . . . 60

9 Comparación de modelos **62**
9.1 Tabla y visualización comparativa . . . 62

10 Aplicaciones económicas **64**
10.1 Aplicación 1: Predicción de precios de vivienda . . . 64
10.2 Aplicación 2: Predicción de ventas . . . 69

11 Ejercicios prácticos **73**
11.1 Ejercicio: Validación cruzada . . . 73

12 Conclusión **74**

13 Próximos pasos **75**

14 Recursos adicionales **75**

15 Publicaciones Similares **75**

Modelos de regresión en Machine Learning

En esta décima guía exploraremos los principales algoritmos de machine learning para problemas de regresión. Estos métodos son fundamentales para resolver problemas económicos como predicción de precios, estimación de demanda, forecasting de series de tiempo, valuación de activos, entre otros.

1 La regresión

1.1 Concepto

La regresión es un método para predecir valores numéricos continuos (a diferencia de clasificación que predice categorías discretas).

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Scikit-learn
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.neighbors import KNeighborsRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# Configuración
plt.style.use('seaborn-v0_8-whitegrid')
plt.rcParams['figure.figsize'] = (12, 6)
np.random.seed(42)

print("Librerías importadas exitosamente")
```

```
Librerías importadas exitosamente
```

1.2 Tipos de problemas de regresión

1. **Predicción de precios**
    - Precios de viviendas
    - Precios de acciones
    - Precios de commodities
    - Tipos de cambio
2. **Estimación de demanda***
    - Demanda de productos
    - Demanda laboral
    - Demanda de servicios públicos

3. **Forecasting económico** (pronósticos)

    - PIB futuro
    - Ventas futuras
    - Inflación
    - Tasas de interés

4. **Valuación**

    - Valuación de empresas
    - Valuación de activos
    - Riesgo de crédito (score continuo)

5. **Análisis de impacto y causalidad**

    - Efecto de políticas públicas
    - Retorno de inversión en publicidad
    - Elasticidades (precio, ingreso)

1.3 Generar datos de ejemplo

```
def generar_datos_regresion(n=200, ruido=1.0, seed=42):
    """
    Genera datos sintéticos para regresión

    Modelo: Y = 3 + 2*X1 - 1.5*X2 +
    """
    np.random.seed(seed)

    # Variables explicativas
    X = np.random.randn(n, 2)

    # Coeficientes verdaderos
    beta_true = np.array([2.0, -1.5])
    intercepto_true = 3.0

    # Variable dependiente con ruido
    Y = intercepto_true + np.dot(X, beta_true) + np.random.randn(n) * ruido

    return X, Y, beta_true, intercepto_true

# Generar datos
X, Y, beta_true, intercepto_true = generar_datos_regresion(n=200, ruido=1.0)

print("DATOS GENERADOS")
print("=" * 70)
print(f"Número de observaciones: {len(Y)}")
print(f"Número de características: {X.shape[1]}")
```

```
print(f"\nCoeficientes verdaderos:")
print(f"  Intercepto: {intercepto_true}")
print(f"   : {beta_true[0]}")
print(f"   : {beta_true[1]}")
print(f"\nModelo verdadero: Y = {intercepto_true} + {beta_true[0]}*X  + {beta_true[1]}

# Visualizar datos
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].scatter(X[:, 0], Y, alpha=0.6, s=50)
axes[0].set_xlabel('X ')
axes[0].set_ylabel('Y')
axes[0].set_title('Relación Y vs X ')
axes[0].grid(True, alpha=0.3)

axes[1].scatter(X[:, 1], Y, alpha=0.6, s=50, color='orange')
axes[1].set_xlabel('X ')
axes[1].set_ylabel('Y')
axes[1].set_title('Relación Y vs X ')
axes[1].grid(True, alpha=0.3)

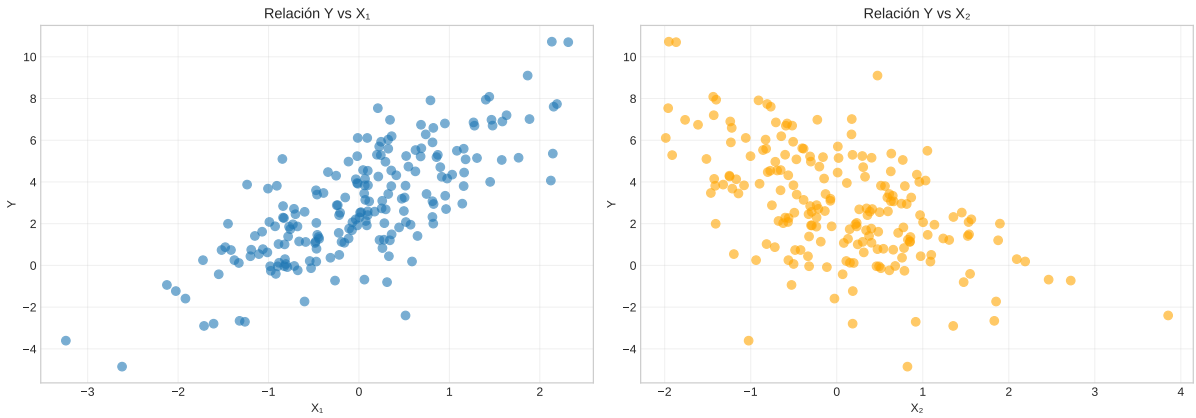
plt.tight_layout()
plt.savefig('datos_regresion.png', dpi=300, bbox_inches='tight')
print("\nGráfico guardado como 'datos_regresion.png'")
```

```
DATOS GENERADOS
================================================================================
Número de observaciones: 200
Número de características: 2

Coeficientes verdaderos:
  Intercepto: 3.0
   : 2.0
   : -1.5

Modelo verdadero: Y = 3.0 + 2.0*X  + -1.5*X  +

Gráfico guardado como 'datos_regresion.png'
```



1.4 Dividir datos

```
# Dividir en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(
    X, Y, test_size=0.3, random_state=42
)

print(f"\nDivisión de datos:")
print(f"  Entrenamiento: {len(y_train)} observaciones")
print(f"  Prueba: {len(y_test)} observaciones")
```

```
División de datos:
  Entrenamiento: 140 observaciones
  Prueba: 60 observaciones
```

2 Mínimos cuadrados ordinarios (OLS)

2.1 Fundamento matemático

El método de Mínimos Cuadrados Ordinarios (Ordinary Least Squares, OLS) es el método más fundamental y ampliamente utilizado en econometría y análisis de regresión. Fue desarrollado por Carl Friedrich Gauss a principios del siglo XIX y constituye la base para muchos métodos estadísticos modernos.

2.1.1 Especificación del modelo

El modelo de regresión lineal múltiple se especifica como:

$$Y_i = \beta_0 + \beta_1 X_{1i} + \beta_2 X_{2i} + \cdots + \beta_p X_{pi} + \varepsilon_i$$

donde:

- Y_i es el valor observado de la variable dependiente para la observación i
- X_{ji} es el valor de la j-ésima variable independiente para la observación i
- β_0 es el intercepto (término constante)
- β_j son los coeficientes de regresión para $j = 1, 2, \ldots, p$
- ε_i es el término de error aleatorio
- $i = 1, 2, \ldots, n$ indexa las observaciones

En notación matricial, el modelo se expresa de forma más compacta como:

$$\mathbf{Y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$$

donde:

- $\mathbf{Y}$ es un vector $n \times 1$ de observaciones de la variable dependiente
- $\mathbf{X}$ es una matriz $n \times (p + 1)$ de variables independientes (incluye columna de unos para el intercepto)
- $\boldsymbol{\beta}$ es un vector $(p + 1) \times 1$ de coeficientes
- $\boldsymbol{\varepsilon}$ es un vector $n \times 1$ de errores

2.1.2 Función objetivo

El método OLS busca estimar los coeficientes $\boldsymbol{\beta}$ que minimizan la Suma de Residuos al Cuadrado (Residual Sum of Squares, RSS):

$$\text{RSS}(\boldsymbol{\beta}) = \sum_{i=1}^{n} \varepsilon_i^2 = \sum_{i=1}^{n} (Y_i - \hat{Y}_i)^2$$

donde $\hat{Y}_i = \beta_0 + \beta_1 X_{1i} + \cdots + \beta_p X_{pi}$ es el valor predicho.
Expandiendo la expresión:

$$\text{RSS}(\boldsymbol{\beta}) = \sum_{i=1}^{n} \left(Y_i - \beta_0 - \sum_{j=1}^{p} \beta_j X_{ji} \right)^2$$

En notación matricial:

$$\text{RSS}(\boldsymbol{\beta}) = (\mathbf{Y} - \mathbf{X}\boldsymbol{\beta})^\top (\mathbf{Y} - \mathbf{X}\boldsymbol{\beta})$$

2.1.3 Derivación de la solución

Para encontrar el estimador que minimiza RSS, tomamos la derivada con respecto a $\boldsymbol{\beta}$ e igualamos a cero:

$$\frac{\partial \text{RSS}}{\partial \boldsymbol{\beta}} = -2\mathbf{X}^\top (\mathbf{Y} - \mathbf{X}\boldsymbol{\beta}) = 0$$

Resolviendo para $\boldsymbol{\beta}$:

$$\mathbf{X}^\top \mathbf{Y} = \mathbf{X}^\top \mathbf{X}\boldsymbol{\beta}$$

Estas son las **ecuaciones normales** del problema de mínimos cuadrados. Asumiendo que $\mathbf{X}^\top \mathbf{X}$ es invertible (es decir, las columnas de $\mathbf{X}$ son linealmente independientes), la solución es:

$$\hat{\boldsymbol{\beta}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{Y}$$

Esta es la **solución de mínimos cuadrados ordinarios**.

2.1.4 Propiedades del estimador OLS

Bajo los supuestos estándar (ver sección siguiente), el estimador OLS posee las siguientes propiedades deseables:

1. **Insesgadez**: El estimador es insesgado, es decir:

$$E[\hat{\boldsymbol{\beta}}] = \boldsymbol{\beta}$$

2. **Varianza mínima**: Según el Teorema de Gauss-Markov, OLS es el estimador lineal insesgado de varianza mínima (BLUE: Best Linear Unbiased Estimator).

3. **Consistencia**: El estimador converge en probabilidad al verdadero valor cuando $n \to \infty$:

$$\text{plim}_{n\to\infty}\hat{\boldsymbol{\beta}} = \boldsymbol{\beta}$$

4. **Normalidad asintótica**: Bajo condiciones regulares:

$$\sqrt{n}(\hat{\boldsymbol{\beta}} - \boldsymbol{\beta}) \xrightarrow{d} N(0, \sigma^2(\mathbf{X}^\top\mathbf{X})^{-1})$$

2.1.5 Matriz de varianzas-covarianzas

La matriz de varianzas-covarianzas del estimador OLS está dada por:

$$\text{Var}(\hat{\boldsymbol{\beta}}) = \sigma^2(\mathbf{X}^\top\mathbf{X})^{-1}$$

donde $\sigma^2 = \text{Var}(\varepsilon_i)$ es la varianza del error. En la práctica, σ^2 se estima mediante:

$$\hat{\sigma}^2 = \frac{\text{RSS}}{n-p-1} = \frac{\sum_{i=1}^{n}\hat{\varepsilon}_i^2}{n-p-1}$$

Los errores estándar de los coeficientes son las raíces cuadradas de los elementos diagonales de $\hat{\sigma}^2(\mathbf{X}^\top\mathbf{X})^{-1}$.

2.1.6 Supuestos del modelo clásico de regresión lineal

Para que el estimador OLS tenga las propiedades óptimas mencionadas, se requieren los siguientes supuestos:

Supuesto 1: Linealidad

El modelo es lineal en los parámetros. La verdadera relación entre Y y las X puede expresarse como:

$$E[Y_i|X_{1i},\ldots,X_{pi}] = \beta_0 + \beta_1 X_{1i} + \cdots + \beta_p X_{pi}$$

Supuesto 2: Exogeneidad estricta

Los errores tienen media condicional cero:

$$E[\varepsilon_i|\mathbf{X}] = 0 \quad \text{para todo } i$$

Esto implica que las variables independientes no están correlacionadas con el término de error.

Supuesto 3: No colinealidad perfecta

Las columnas de $\mathbf{X}$ son linealmente independientes, es decir, la matriz $\mathbf{X}^\top\mathbf{X}$ es de rango completo y por tanto invertible. No existe $\mathbf{a} \neq \mathbf{0}$ tal que $\mathbf{Xa} = \mathbf{0}$.

Supuesto 4: Homocedasticidad
Los errores tienen varianza constante:

$$\text{Var}(\varepsilon_i|\mathbf{X}) = \sigma^2 \quad \text{para todo } i$$

Supuesto 5: No autocorrelación
Los errores no están correlacionados entre sí:

$$\text{Cov}(\varepsilon_i, \varepsilon_j|\mathbf{X}) = 0 \quad \text{para } i \neq j$$

Los supuestos 4 y 5 pueden combinarse como:

$$E[\boldsymbol{\varepsilon}\boldsymbol{\varepsilon}^\top|\mathbf{X}] = \sigma^2\mathbf{I}_n$$

Supuesto 6: Normalidad (opcional)
Para inferencia en muestras finitas, se asume:

$$\varepsilon_i|\mathbf{X} \sim N(0, \sigma^2)$$

Este supuesto no es necesario para propiedades asintóticas pero permite realizar pruebas de hipótesis exactas en muestras pequeñas.

2.1.7 Ventajas del método OLS

- **Simplicidad conceptual e implementación**: El método tiene una interpretación geométrica clara (proyección ortogonal) y es computacionalmente eficiente.
- **Interpretabilidad**: Los coeficientes tienen interpretación directa como efectos marginales. β_j representa el cambio en Y por un cambio unitario en X_j, manteniendo constantes las demás variables.
- **Solución cerrada**: Existe una fórmula analítica explícita para el estimador, lo que facilita el análisis teórico.
- **Propiedades estadísticas óptimas**: Bajo los supuestos del modelo clásico, OLS es BLUE.
- **Base para extensiones**: OLS es el fundamento de métodos más sofisticados como mínimos cuadrados generalizados (GLS), variables instrumentales (IV), y modelos de panel.
- **Inferencia estadística**: Permite construir intervalos de confianza y realizar pruebas de hipótesis sobre los coeficientes.

2.1.8 Limitaciones del método OLS

- **Sensibilidad a violaciones de supuestos**:
  - La **heterocedasticidad** (varianza no constante) invalida los errores estándar estándar
  - La **autocorrelación** (común en series de tiempo) sesga los errores estándar
  - La **endogeneidad** (correlación entre X y ε) produce estimadores sesgados e inconsistentes

- **Sensibilidad a outliers**: Valores extremos pueden ejercer influencia desproporcionada en las estimaciones debido a la minimización de errores al cuadrado.
- **Multicolinealidad**: Cuando las variables independientes están altamente correlacionadas:
  - Los errores estándar de los coeficientes se inflan
  - Los coeficientes individuales se vuelven inestables
  - La matriz $\mathbf{X}^\top\mathbf{X}$ se aproxima a singularidad
- **Restricción a modelos lineales**: OLS solo puede capturar relaciones lineales en los parámetros (aunque puede incluir transformaciones no lineales de las variables).
- **No realiza selección de variables**: Incluye todas las variables especificadas sin discriminar su relevancia.
- **Overfitting con muchas variables**: En modelos con p grande relativo a n, puede ajustar demasiado al ruido de los datos de entrenamiento, deteriorando la capacidad predictiva fuera de muestra.

2.1.9 Aplicaciones en economía

El método OLS es ubicuo en economía y se aplica a:

- **Estimación de funciones de demanda**: Relación entre cantidad demandada, precio, ingreso y otros factores
- **Funciones de producción**: Relación entre insumos (capital, trabajo) y producto
- **Ecuaciones de salarios**: Retornos a la educación y experiencia (ecuación de Mincer)
- **Modelos de consumo**: Propensión marginal a consumir
- **Análisis de series macroeconómicas**: PIB, inflación, desempleo
- **Evaluación de políticas**: Efectos de programas sociales o intervenciones

2.1.10 Consideraciones computacionales

La inversión de la matriz $\mathbf{X}^\top\mathbf{X}$ es computacionalmente costosa cuando p es grande. Métodos numéricos estables incluyen:

- **Descomposición QR**: $\mathbf{X} = \mathbf{QR}$, luego $\hat{\boldsymbol{\beta}} = \mathbf{R}^{-1}\mathbf{Q}^\top\mathbf{Y}$
- **Descomposición SVD**: Descomposición en valores singulares de $\mathbf{X}$
- **Eliminación gaussiana**: Para resolver directamente las ecuaciones normales

En Python, `sklearn.linear_model.LinearRegression` utiliza descomposición SVD por defecto, lo que proporciona estabilidad numérica superior.

2.2 Implementación

```
# Entrenar modelo OLS
modelo_ols = LinearRegression()
modelo_ols.fit(X_train, y_train)

# Coeficientes estimados
```

```
print("\nMODELO OLS ESTIMADO")
print("=" * 70)
print(f"Intercepto estimado: {modelo_ols.intercept_:.4f}")
print(f"  (Verdadero: {intercepto_true})")
print(f"\nCoeficientes estimados:")
print(f"   : {modelo_ols.coef_[0]:.4f} (Verdadero: {beta_true[0]})")
print(f"   : {modelo_ols.coef_[1]:.4f} (Verdadero: {beta_true[1]})")

# Predecir
y_pred_ols_train = modelo_ols.predict(X_train)
y_pred_ols_test = modelo_ols.predict(X_test)

# Evaluar
r2_train = r2_score(y_train, y_pred_ols_train)
r2_test = r2_score(y_test, y_pred_ols_test)
mse_train = mean_squared_error(y_train, y_pred_ols_train)
mse_test = mean_squared_error(y_test, y_pred_ols_test)

print(f"\n\nPERFORMANCE DEL MODELO")
print("=" * 70)
print(f"R² Train: {r2_train:.4f}")
print(f"R² Test:  {r2_test:.4f}")
print(f"MSE Train: {mse_train:.4f}")
print(f"MSE Test:  {mse_test:.4f}")
```

```
MODELO OLS ESTIMADO
======================================================================
Intercepto estimado: 2.8623
  (Verdadero: 3.0)

Coeficientes estimados:
   : 2.1145 (Verdadero: 2.0)
   : -1.3336 (Verdadero: -1.5)


PERFORMANCE DEL MODELO
======================================================================
R² Train: 0.8619
R² Test:  0.8480
MSE Train: 0.9534
MSE Test:  1.0902
```

2.3 Visualización de predicciones

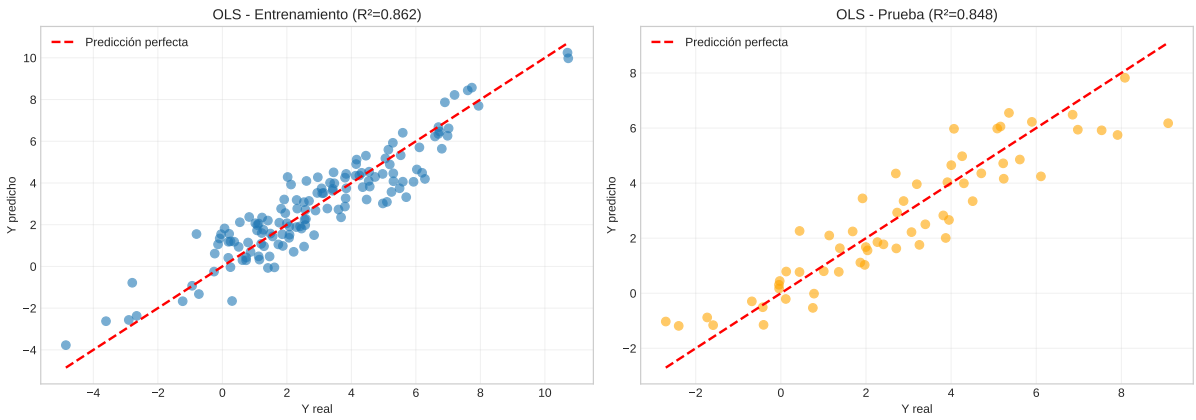
```
# Gráfico de valores reales vs predichos
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
```

```
# Training set
axes[0].scatter(y_train, y_pred_ols_train, alpha=0.6, s=50)
axes[0].plot([y_train.min(), y_train.max()],
             [y_train.min(), y_train.max()],
             'r--', linewidth=2, label='Predicción perfecta')
axes[0].set_xlabel('Y real')
axes[0].set_ylabel('Y predicho')
axes[0].set_title(f'OLS - Entrenamiento (R²={r2_train:.3f})')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Test set
axes[1].scatter(y_test, y_pred_ols_test, alpha=0.6, s=50, color='orange')
axes[1].plot([y_test.min(), y_test.max()],
             [y_test.min(), y_test.max()],
             'r--', linewidth=2, label='Predicción perfecta')
axes[1].set_xlabel('Y real')
axes[1].set_ylabel('Y predicho')
axes[1].set_title(f'OLS - Prueba (R²={r2_test:.3f})')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('ols_predictions.png', dpi=300, bbox_inches='tight')
print("\nGráfico guardado como 'ols_predictions.png'")
```

```
Gráfico guardado como 'ols_predictions.png'
```



2.4 Análisis de residuos

```
# Calcular residuos
residuos_train = y_train - y_pred_ols_train
residuos_test = y_test - y_pred_ols_test

# Gráficos de diagnóstico
```

```
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# 1. Residuos vs valores predichos
axes[0, 0].scatter(y_pred_ols_train, residuos_train, alpha=0.6)
axes[0, 0].axhline(y=0, color='r', linestyle='--', linewidth=2)
axes[0, 0].set_xlabel('Valores predichos')
axes[0, 0].set_ylabel('Residuos')
axes[0, 0].set_title('Residuos vs Predicciones')
axes[0, 0].grid(True, alpha=0.3)

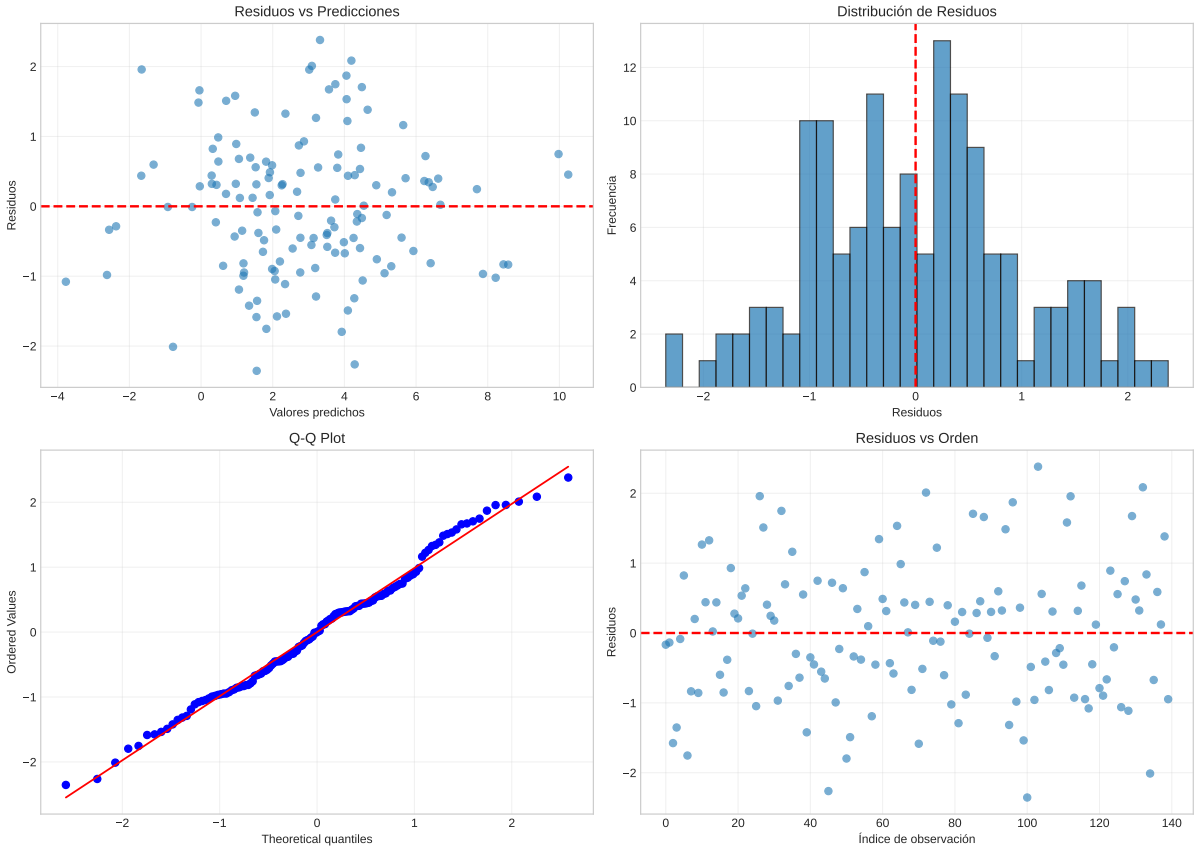
# 2. Histograma de residuos
axes[0, 1].hist(residuos_train, bins=30, edgecolor='black', alpha=0.7)
axes[0, 1].axvline(x=0, color='r', linestyle='--', linewidth=2)
axes[0, 1].set_xlabel('Residuos')
axes[0, 1].set_ylabel('Frecuencia')
axes[0, 1].set_title('Distribución de Residuos')
axes[0, 1].grid(True, alpha=0.3)

# 3. Q-Q plot
from scipy import stats
stats.probplot(residuos_train, dist="norm", plot=axes[1, 0])
axes[1, 0].set_title('Q-Q Plot')
axes[1, 0].grid(True, alpha=0.3)

# 4. Residuos vs orden
axes[1, 1].scatter(range(len(residuos_train)), residuos_train, alpha=0.6)
axes[1, 1].axhline(y=0, color='r', linestyle='--', linewidth=2)
axes[1, 1].set_xlabel('Índice de observación')
axes[1, 1].set_ylabel('Residuos')
axes[1, 1].set_title('Residuos vs Orden')
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('ols_diagnostics.png', dpi=300, bbox_inches='tight')
print("Gráficos de diagnóstico guardados como 'ols_diagnostics.png'")
```

```
Gráficos de diagnóstico guardados como 'ols_diagnostics.png'
```



3 Regresión Ridge

3.1 Fundamento matemático

La regresión Ridge, también conocida como regularización L2 o regresión de cresta, fue propuesta por Hoerl y Kennard (1970) como una solución al problema de multicolinealidad en regresión lineal. Este método introduce una penalización sobre la magnitud de los coeficientes para estabilizar las estimaciones.

3.1.1 Especificación del problema

La regresión Ridge modifica el problema de mínimos cuadrados ordinarios añadiendo un término de penalización:

$$\min_{\boldsymbol{\beta}} \left\{ \text{RSS}(\boldsymbol{\beta}) + \alpha \|\boldsymbol{\beta}\|_2^2 \right\}$$

donde:

- $\text{RSS}(\boldsymbol{\beta}) = \sum_{i=1}^{n} (Y_i - \mathbf{X}_i^\top \boldsymbol{\beta})^2$ es la suma de residuos al cuadrado
- $\|\boldsymbol{\beta}\|_2^2 = \sum_{j=1}^{p} \beta_j^2$ es la norma L2 al cuadrado de los coeficientes
- $\alpha \geq 0$ es el parámetro de regularización que controla la intensidad de la penalización

Expandiendo la función objetivo:

$$\mathcal{L}(\boldsymbol{\beta}) = \sum_{i=1}^{n} \left(Y_i - \beta_0 - \sum_{j=1}^{p} \beta_j X_{ji} \right)^2 + \alpha \sum_{j=1}^{p} \beta_j^2$$

En notación matricial:

$$\mathcal{L}(\boldsymbol{\beta}) = (\mathbf{Y} - \mathbf{X}\boldsymbol{\beta})^\top (\mathbf{Y} - \mathbf{X}\boldsymbol{\beta}) + \alpha \boldsymbol{\beta}^\top \boldsymbol{\beta}$$

Nota importante: Generalmente, el intercepto β_0 no se penaliza, es decir, la penalización se aplica solo a $\beta_1, \ldots, \beta_p$. Esto se debe a que el intercepto simplemente centra los datos y no debería ser objeto de regularización.

3.1.2 Derivación de la solución

Para encontrar el estimador Ridge, derivamos la función objetivo con respecto a $\boldsymbol{\beta}$ e igualamos a cero:

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{\beta}} = -2\mathbf{X}^\top (\mathbf{Y} - \mathbf{X}\boldsymbol{\beta}) + 2\alpha \boldsymbol{\beta} = 0$$

Reordenando:

$$\mathbf{X}^\top \mathbf{Y} = \mathbf{X}^\top \mathbf{X}\boldsymbol{\beta} + \alpha \boldsymbol{\beta}$$

$$\mathbf{X}^\top \mathbf{Y} = (\mathbf{X}^\top \mathbf{X} + \alpha \mathbf{I})\boldsymbol{\beta}$$

Por lo tanto, la **solución de regresión Ridge** es:

$$\hat{\boldsymbol{\beta}}^{\text{Ridge}} = (\mathbf{X}^\top \mathbf{X} + \alpha \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{Y}$$

donde $\mathbf{I}$ es la matriz identidad de dimensión $p \times p$.

3.1.3 Interpretación geométrica

La solución Ridge puede interpretarse como el resultado de añadir α a los elementos diagonales de $\mathbf{X}^\top \mathbf{X}$, lo que garantiza que la matriz sea invertible incluso cuando $\mathbf{X}^\top \mathbf{X}$ es singular o casi singular (problema de multicolinealidad).

3.1.4 Efecto del parámetro de regularización α

El parámetro α controla el trade-off entre ajuste a los datos y complejidad del modelo:
Caso $\alpha = 0$:

$$\hat{\boldsymbol{\beta}}^{\text{Ridge}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{Y} = \hat{\boldsymbol{\beta}}^{\text{OLS}}$$

La solución es idéntica a OLS (sin regularización).

Caso $\alpha \to \infty$:

$$\lim_{\alpha \to \infty} \hat{\boldsymbol{\beta}}^{\text{Ridge}} = \mathbf{0}$$

Los coeficientes se reducen a cero (máxima regularización).

Casos intermedios ($0 < \alpha < \infty$): Los coeficientes se "encogen" (shrinkage) hacia cero, pero nunca alcanzan exactamente cero.

3.1.5 Propiedades del estimador Ridge

1. Sesgo del estimador
A diferencia de OLS, el estimador Ridge es sesgado:

$$E[\hat{\boldsymbol{\beta}}^{\text{Ridge}}] = (\mathbf{X}^\top\mathbf{X} + \alpha\mathbf{I})^{-1}\mathbf{X}^\top\mathbf{X}\boldsymbol{\beta} \neq \boldsymbol{\beta}$$

El sesgo es:

$$\text{Bias}[\hat{\boldsymbol{\beta}}^{\text{Ridge}}] = -\alpha(\mathbf{X}^\top\mathbf{X} + \alpha\mathbf{I})^{-1}\boldsymbol{\beta}$$

2. Varianza del estimador
La matriz de varianzas-covarianzas es:

$$\text{Var}(\hat{\boldsymbol{\beta}}^{\text{Ridge}}) = \sigma^2(\mathbf{X}^\top\mathbf{X} + \alpha\mathbf{I})^{-1}\mathbf{X}^\top\mathbf{X}(\mathbf{X}^\top\mathbf{X} + \alpha\mathbf{I})^{-1}$$

que es generalmente menor que la varianza de OLS:

$$\text{Var}(\hat{\boldsymbol{\beta}}^{\text{OLS}}) = \sigma^2(\mathbf{X}^\top\mathbf{X})^{-1}$$

3. Error cuadrático medio (MSE)
El MSE del estimador Ridge es:

$$\text{MSE}[\hat{\boldsymbol{\beta}}^{\text{Ridge}}] = \text{Var}(\hat{\boldsymbol{\beta}}^{\text{Ridge}}) + \text{Bias}^2[\hat{\boldsymbol{\beta}}^{\text{Ridge}}]$$

Aunque Ridge introduce sesgo, la reducción en varianza puede ser suficiente para que el MSE total sea menor que el de OLS, especialmente en presencia de multicolinealidad.

3.1.6 Trade-off sesgo-varianza

La regresión Ridge ilustra el clásico trade-off entre sesgo y varianza:

- **Aumentar α**: Incrementa el sesgo pero reduce la varianza
- **Disminuir α**: Reduce el sesgo pero incrementa la varianza

Existe un valor óptimo de α que minimiza el error de predicción esperado:

$$\alpha^* = \arg\min_{\alpha} E[(Y_{\text{nuevo}} - \mathbf{X}_{\text{nuevo}}^\top\hat{\boldsymbol{\beta}}^{\text{Ridge}})^2]$$

Este valor óptimo se determina típicamente mediante validación cruzada.

3.1.7 Interpretación Bayesiana

La regresión Ridge tiene una interpretación elegante desde la perspectiva Bayesiana. Es equivalente a estimación de máxima probabilidad a posteriori (MAP) con una distribución a priori gaussiana sobre los coeficientes:

$$\beta_j \sim N(0, \tau^2) \quad \text{donde} \quad \tau^2 = \frac{\sigma^2}{\alpha}$$

La penalización L2 corresponde al logaritmo negativo de esta distribución a priori.

3.1.8 Ventajas de la regresión Ridge

1. **Estabilidad ante multicolinealidad**: Cuando $\mathbf{X}^\top\mathbf{X}$ es casi singular (multicolinealidad alta), OLS produce coeficientes inestables con varianzas muy altas. Ridge estabiliza las estimaciones.

2. **Reducción de varianza**: Especialmente beneficioso cuando p es grande relativo a n o cuando las variables están correlacionadas.

3. **Prevención de overfitting**: La penalización limita la complejidad del modelo, mejorando la generalización.

4. **Solución única garantizada**: La matriz $\mathbf{X}^\top\mathbf{X} + \alpha\mathbf{I}$ siempre es invertible para $\alpha > 0$.

5. **Todas las variables permanecen**: A diferencia de Lasso, Ridge no elimina variables completamente, lo cual es deseable cuando todas las variables son potencialmente relevantes.

6. **Propiedades de grupos**: Cuando hay grupos de variables correlacionadas, Ridge tiende a asignarles coeficientes similares, lo cual puede ser deseable en ciertos contextos económicos.

3.1.9 Limitaciones de la regresión Ridge

1. **No realiza selección de variables**: Todos los coeficientes son distintos de cero (aunque pequeños). Esto puede dificultar la interpretación cuando p es muy grande.

2. **Sesgo introducido**: El estimador es sesgado, lo que puede ser problemático para inferencia causal.

3. **Difícil interpretación**: Los coeficientes encogidos son más difíciles de interpretar que los de OLS.

4. **Elección de α**: Requiere seleccionar el parámetro de regularización, típicamente mediante validación cruzada, lo cual añade complejidad computacional.

5. **Sensibilidad a la escala**: Los resultados dependen de la escala de las variables, por lo que es esencial estandarizar las variables antes de aplicar Ridge.

3.1.10 Estandarización de variables

Antes de aplicar Ridge, es crucial estandarizar las variables independientes:

$$\tilde{X}_{ji} = \frac{X_{ji} - \bar{X}_j}{s_j}$$

donde $\bar{X}_j$ y s_j son la media y desviación estándar de la variable j. Esto asegura que todas las variables contribuyan equitativamente a la penalización.

3.1.11 Aplicaciones en economía

La regresión Ridge es particularmente útil en:

1. **Modelos macroeconómicos**: Con múltiples indicadores correlacionados (PIB, consumo, inversión, etc.)
2. **Predicción de series de tiempo**: Con múltiples rezagos de la variable dependiente y otras variables explicativas
3. **Modelos de pricing**: En finanzas, cuando múltiples factores de riesgo están correlacionados
4. **Análisis de panel**: Con muchos efectos fijos o controles correlacionados
5. **Modelos de valoración**: Cuando hay múltiples características correlacionadas (ej: valoración de viviendas con características de ubicación correlacionadas)
6. **Forecasting económico**: Cuando se incluyen muchos predictores potencialmente relevantes y correlacionados

3.1.12 Selección del parámetro α mediante validación cruzada

El procedimiento estándar para seleccionar α es:

1. Dividir datos en K folds
2. Para cada valor candidato de α:
   - Entrenar modelo en K-1 folds
   - Predecir en el fold restante
   - Calcular error de predicción
3. Promediar errores sobre los K folds
4. Seleccionar α que minimiza el error promedio

Matemáticamente:

$$\alpha^* = \arg\min_{\alpha} \frac{1}{K} \sum_{k=1}^{K} \text{MSE}_k(\alpha)$$

donde $\text{MSE}_k(\alpha)$ es el error cuadrático medio en el k-ésimo fold.

3.2 Implementación

```
# Probar diferentes valores de alpha
alphas = [0.001, 0.01, 0.1, 1, 10, 100, 1000]

print("\nREGRESIÓN RIDGE - EFECTO DE  ")
print("=" * 70)
print(f"{' ':<10} {' ':<12} {' ':<12} {'R² Train':<12} {'R² Test':<12}")
print("-" * 70)
```

```
resultados_ridge = []

for alpha in alphas:
    # Entrenar modelo
    modelo_ridge = Ridge(alpha=alpha)
    modelo_ridge.fit(X_train, y_train)

    # Evaluar
    r2_train = modelo_ridge.score(X_train, y_train)
    r2_test = modelo_ridge.score(X_test, y_test)

    resultados_ridge.append({
        'alpha': alpha,
        'beta1': modelo_ridge.coef_[0],
        'beta2': modelo_ridge.coef_[1],
        'r2_train': r2_train,
        'r2_test': r2_test
    })

    print(f"{alpha:<10.3f} {modelo_ridge.coef_[0]:<12.4f} "
          f"{modelo_ridge.coef_[1]:<12.4f} {r2_train:<12.4f} {r2_test:<12.4f}")

df_ridge = pd.DataFrame(resultados_ridge)
```

```
REGRESIÓN RIDGE - EFECTO DE
======================================================================
                                    R² Train     R² Test
----------------------------------------------------------------------
0.001      2.1145       -1.3336      0.8619       0.8480
0.010      2.1143       -1.3335      0.8619       0.8480
0.100      2.1129       -1.3326      0.8619       0.8480
1.000      2.0986       -1.3239      0.8619       0.8476
10.000     1.9659       -1.2421      0.8577       0.8407
100.000    1.2046       -0.7678      0.7036       0.6801
1000.000   0.2473       -0.1592      0.1909       0.1821
```

3.3 Visualización del efecto de regularización

```
# Gráfico de coeficientes vs alpha
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Coeficientes
axes[0].semilogx(df_ridge['alpha'], df_ridge['beta1'],
                 'o-', linewidth=2, markersize=8, label=' ')
axes[0].semilogx(df_ridge['alpha'], df_ridge['beta2'],
                 's-', linewidth=2, markersize=8, label=' ')
axes[0].axhline(y=beta_true[0], color='blue', linestyle='--',
```

```
                alpha=0.5, label='  verdadero')
axes[0].axhline(y=beta_true[1], color='orange', linestyle='--',
                alpha=0.5, label='  verdadero')
axes[0].set_xlabel('  (escala logarítmica)')
axes[0].set_ylabel('Valor del coeficiente')
axes[0].set_title('Coeficientes vs Parámetro de Regularización')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

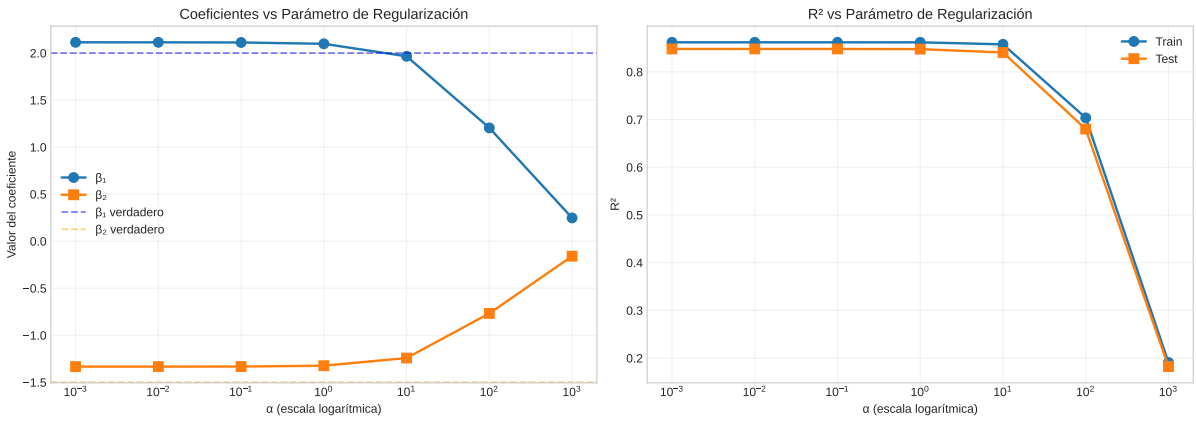
# R²
axes[1].semilogx(df_ridge['alpha'], df_ridge['r2_train'],
                 'o-', linewidth=2, markersize=8, label='Train')
axes[1].semilogx(df_ridge['alpha'], df_ridge['r2_test'],
                 's-', linewidth=2, markersize=8, label='Test')
axes[1].set_xlabel('  (escala logarítmica)')
axes[1].set_ylabel('R²')
axes[1].set_title('R² vs Parámetro de Regularización')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('ridge_regularization.png', dpi=300, bbox_inches='tight')
print("\nGráfico guardado como 'ridge_regularization.png'")

# Mejor alpha
mejor_idx = df_ridge['r2_test'].idxmax()
mejor_alpha = df_ridge.loc[mejor_idx, 'alpha']
print(f"\nMejor  : {mejor_alpha}")
print(f"R² Test: {df_ridge.loc[mejor_idx, 'r2_test']:.4f}")
```

```
Gráfico guardado como 'ridge_regularization.png'

Mejor  : 0.001
R² Test: 0.8480
```



4 Regresión Lasso

4.1 Fundamento matemático

La regresión Lasso (Least Absolute Shrinkage and Selection Operator) fue introducida por Tibshirani (1996) como un método de regularización que simultáneamente realiza selección de variables y regularización de coeficientes. A diferencia de Ridge, Lasso puede reducir algunos coeficientes exactamente a cero, proporcionando modelos más interpretables y parsimoniosos.

4.1.1 Especificación del problema

La regresión Lasso minimiza la siguiente función objetivo:

$$\min_{\boldsymbol{\beta}} \{\text{RSS}(\boldsymbol{\beta}) + \alpha \|\boldsymbol{\beta}\|_1\}$$

donde:

- $\text{RSS}(\boldsymbol{\beta}) = \sum_{i=1}^{n} (Y_i - \mathbf{X}_i^\top \boldsymbol{\beta})^2$ es la suma de residuos al cuadrado
- $\|\boldsymbol{\beta}\|_1 = \sum_{j=1}^{p} |\beta_j|$ es la norma L1 de los coeficientes
- $\alpha \geq 0$ es el parámetro de regularización

Expandiendo:

$$\mathcal{L}(\boldsymbol{\beta}) = \sum_{i=1}^{n} \left(Y_i - \beta_0 - \sum_{j=1}^{p} \beta_j X_{ji} \right)^2 + \alpha \sum_{j=1}^{p} |\beta_j|$$

En notación matricial:

$$\mathcal{L}(\boldsymbol{\beta}) = (\mathbf{Y} - \mathbf{X}\boldsymbol{\beta})^\top (\mathbf{Y} - \mathbf{X}\boldsymbol{\beta}) + \alpha \|\boldsymbol{\beta}\|_1$$

Nota importante: Al igual que en Ridge, el intercepto β_0 típicamente no se penaliza.

4.1.2 Diferencia fundamental con Ridge

La diferencia clave entre Lasso y Ridge radica en la norma utilizada:
Ridge (L2):

$$\text{Penalización} = \alpha \sum_{j=1}^{p} \beta_j^2$$

Lasso (L1):

$$\text{Penalización} = \alpha \sum_{j=1}^{p} |\beta_j|$$

La norma L1 es la suma de valores absolutos, mientras que la norma L2 es la suma de cuadrados. Esta diferencia aparentemente sutil tiene consecuencias profundas.

4.1.3 Solución del problema Lasso

A diferencia de Ridge y OLS, Lasso **no tiene solución analítica cerrada** debido a la no diferenciabilidad de la función valor absoluto en cero. La función $|\beta_j|$ no es diferenciable en $\beta_j = 0$.

La solución debe obtenerse mediante métodos de optimización numérica:

1. Coordinate Descent (Descenso por coordenadas)

Actualiza un coeficiente a la vez, manteniendo los demás fijos. Para cada coeficiente β_j:

$$\hat{\beta}_j = \text{sign}(\tilde{\beta}_j^{\text{OLS}}) \cdot \max(0, |\tilde{\beta}_j^{\text{OLS}}| - \alpha)$$

donde $\tilde{\beta}_j^{\text{OLS}}$ es el coeficiente OLS condicional en los demás coeficientes. Esta es la operación de **soft-thresholding**.

2. LARS (Least Angle Regression)

Algoritmo eficiente que construye el path completo de soluciones Lasso para todos los valores de α.

3. Proximal Gradient Descent

Métodos de optimización convexa que explotan la estructura del problema.

4.1.4 Interpretación geométrica

La diferencia en el comportamiento de Ridge y Lasso puede entenderse geométricamente:

Formulación equivalente con restricción:

Ridge:

$$\min_{\boldsymbol{\beta}} \text{RSS}(\boldsymbol{\beta}) \quad \text{sujeto a} \quad \sum_{j=1}^{p} \beta_j^2 \leq t$$

Lasso:

$$\min_{\boldsymbol{\beta}} \text{RSS}(\boldsymbol{\beta}) \quad \text{sujeto a} \quad \sum_{j=1}^{p} |\beta_j| \leq t$$

Donde existe una correspondencia uno a uno entre α y t.

Región de restricción:

- Ridge: La restricción $\sum \beta_j^2 \leq t$ define una **hiperesfera** (bola)
- Lasso: La restricción $\sum |\beta_j| \leq t$ define un **hiperrombo** (diamante)

La solución óptima ocurre donde las curvas de nivel de RSS tocan primero la región de restricción.

Punto clave: El hiperrombo de Lasso tiene "esquinas" en los ejes donde algunos $\beta_j = 0$. Es más probable que las curvas de nivel de RSS toquen estas esquinas, lo que explica por qué Lasso produce coeficientes exactamente cero.

4.1.5 Propiedades de soft-thresholding

La operación de soft-thresholding es fundamental en Lasso:

$$\mathcal{S}_\alpha(\beta) = \begin{cases} \beta - \alpha & \text{si } \beta > \alpha \\ 0 & \text{si } |\beta| \leq \alpha \\ \beta + \alpha & \text{si } \beta < -\alpha \end{cases}$$

Esta función: 1. **Encoge** coeficientes hacia cero 2. **Establece en cero** coeficientes pequeños (menores que α en valor absoluto) 3. Es la operación de proximidad para la norma L1

4.1.6 Efecto del parámetro α

Caso $\alpha = 0$: Sin penalización, se reduce a OLS (si las variables están estandarizadas apropiadamente).

Caso α pequeño: Pocos coeficientes se establecen exactamente en cero. El modelo incluye casi todas las variables.

Caso α moderado: Balance entre ajuste y parsimonia. Algunos coeficientes son cero (selección de variables).

Caso α grande: Muchos o todos los coeficientes son cero. Modelo muy regularizado.

Caso $\alpha \to \infty$:

$$\lim_{\alpha \to \infty} \hat{\boldsymbol{\beta}}^{\text{Lasso}} = \mathbf{0}$$

Todos los coeficientes se reducen a cero.

4.1.7 Propiedades del estimador Lasso

1. Selección de variables

La propiedad más distintiva de Lasso es que produce **modelos sparse** (dispersos):

$$\hat{\beta}_j^{\text{Lasso}} = 0 \quad \text{para algunos } j$$

Esto proporciona selección automática de variables, identificando las variables más relevantes.

2. Sesgo y varianza

Similar a Ridge, Lasso introduce sesgo pero reduce varianza:

$$E[\hat{\boldsymbol{\beta}}^{\text{Lasso}}] \neq \boldsymbol{\beta}$$

El trade-off sesgo-varianza se controla mediante α.

3. Consistencia en selección

Bajo ciertas condiciones (irrepresentability condition), Lasso identifica consistentemente el conjunto verdadero de variables relevantes cuando $n \to \infty$.

4. Comportamiento con variables correlacionadas

Cuando variables están altamente correlacionadas, Lasso tiende a:

- Seleccionar arbitrariamente una del grupo
- Ignorar las demás

Este comportamiento puede ser indeseable si todas las variables correlacionadas son relevantes.

4.1.8 Comparación Ridge vs Lasso

Característica	Ridge (L2)	Lasso (L1)
Penalización	$\sum \beta_j^2$	$\sum \|\beta_j\|$
Solución	Analítica cerrada	Numérica
Coeficientes cero	No (solo encoge)	Sí (selección)

Característica	Ridge (L2)	Lasso (L1)
Variables correlacionadas	Coeficientes similares	Selecciona una
Interpretabilidad	Menos (todas las variables)	Más (modelo sparse)
Estabilidad	Más estable	Menos estable
Uso principal	Reducir varianza	Selección de variables

4.1.9 Ventajas de la regresión Lasso

1. **Selección automática de variables**: Identifica las variables más importantes, estableciendo otras en cero.

2. **Modelos interpretables**: Produce modelos parsimoniosos más fáciles de entender y comunicar.

3. **Prevención de overfitting**: Especialmente útil cuando $p >> n$ (más variables que observaciones).

4. **Reducción de varianza**: Similar a Ridge, reduce la varianza del modelo.

5. **Útil para identificación**: En economía, ayuda a identificar los determinantes clave de un fenómeno.

6. **Computational efficiency**: Los algoritmos modernos (coordinate descent) son muy eficientes.

4.1.10 Limitaciones de la regresión Lasso

1. **Selección arbitraria con correlación**: Con variables correlacionadas, puede seleccionar arbitrariamente entre ellas, lo cual puede no ser deseable.

2. **Máximo p < n variables**: En el caso extremo donde $p > n$, Lasso puede seleccionar como máximo n variables (limitación del algoritmo).

3. **Inestabilidad**: Pequeños cambios en los datos pueden cambiar qué variables son seleccionadas.

4. **No agrupa variables correlacionadas**: A diferencia de Ridge, no asigna coeficientes similares a variables correlacionadas.

5. **Sesgo introducido**: Especialmente problemático para coeficientes grandes que deberían permanecer grandes.

6. **Elección de α**: Requiere validación cruzada, añadiendo complejidad computacional.

4.1.11 Interpretación Bayesiana

Lasso corresponde a MAP estimation con una distribución a priori de Laplace (doble exponencial):

$$p(\beta_j) = \frac{1}{2b} \exp\left(-\frac{|\beta_j|}{b}\right)$$

donde $b = \frac{\sigma^2}{\alpha}$. Esta distribución tiene más masa de probabilidad en cero que la distribución gaussiana (a priori de Ridge), explicando por qué Lasso produce más coeficientes exactamente cero.

4.1.12 Estandarización de variables

Al igual que Ridge, es **crucial** estandarizar variables antes de aplicar Lasso:

$$\tilde{X}_{ji} = \frac{X_{ji} - \bar{X}_j}{s_j}$$

Sin estandarización, la penalización L1 penalizaría desproporcionadamente variables con escalas grandes.

4.1.13 Aplicaciones en economía

Lasso es particularmente valioso en:

1. **Variable selection en modelos empíricos**: Identificar determinantes clave de outcomes económicos entre muchos candidatos.

2. **Modelos de forecasting**: Seleccionar predictores relevantes de series macroeconómicas.

3. **Análisis de text mining**: Identificar palabras clave en análisis de sentimiento económico.

4. **Modelos de credit scoring**: Seleccionar características predictivas de default.

5. **Economía del desarrollo**: Identificar factores clave de desarrollo entre múltiples variables institucionales, geográficas y económicas.

6. **Finanzas**: Selección de factores relevantes en modelos de pricing de activos.

7. **High-dimensional econometrics**: Modelos con muchas variables potenciales y relativamente pocas observaciones.

4.1.14 Selección del parámetro α

El procedimiento estándar es validación cruzada:

$$\alpha^* = \arg\min_{\alpha} \frac{1}{K} \sum_{k=1}^{K} \text{MSE}_k(\alpha)$$

Regla de un error estándar: En la práctica, se suele seleccionar el α más grande (más regularización) cuyo error esté dentro de un error estándar del mínimo:

$$\alpha^{1SE} = \max\{\alpha : \text{MSE}(\alpha) \leq \text{MSE}(\alpha^*) + \text{SE}(\alpha^*)\}$$

Esto favorece modelos más simples cuando el error de predicción es comparable.

4.1.15 Lasso adaptativo

Una extensión importante es el **Adaptive Lasso**, que usa pesos diferentes para cada coeficiente:

$$\mathcal{L}(\boldsymbol{\beta}) = \text{RSS}(\boldsymbol{\beta}) + \alpha \sum_{j=1}^{p} w_j |\beta_j|$$

donde típicamente $w_j = 1/|\hat{\beta}_j^{\text{OLS}}|^\gamma$ con $\gamma > 0$. Esto mejora las propiedades de selección consistente del modelo.

4.2 Implementación

```
# Probar diferentes valores de alpha
alphas_lasso = [0.001, 0.01, 0.1, 0.5, 1, 2, 5]

print("\nREGRESIÓN LASSO - EFECTO DE  ")
print("=" * 80)
print(f"{' ':<10} {' ':<12} {' ':<12} {'Variables 0':<14} {'R² Train':<12} {'R² Test':
print("-" * 80)

resultados_lasso = []

for alpha in alphas_lasso:
    # Entrenar modelo
    modelo_lasso = Lasso(alpha=alpha, max_iter=10000)
    modelo_lasso.fit(X_train, y_train)

    # Contar variables no cero
    n_nonzero = np.sum(modelo_lasso.coef_ != 0)

    # Evaluar
    r2_train = modelo_lasso.score(X_train, y_train)
    r2_test = modelo_lasso.score(X_test, y_test)

    resultados_lasso.append({
        'alpha': alpha,
        'beta1': modelo_lasso.coef_[0],
        'beta2': modelo_lasso.coef_[1],
        'n_nonzero': n_nonzero,
        'r2_train': r2_train,
        'r2_test': r2_test
    })

    print(f"{alpha:<10.3f} {modelo_lasso.coef_[0]:<12.4f} "
          f"{modelo_lasso.coef_[1]:<12.4f} {n_nonzero:<14d} "
          f"{r2_train:<12.4f} {r2_test:<12.4f}")

df_lasso = pd.DataFrame(resultados_lasso)
```

```
REGRESIÓN LASSO - EFECTO DE
================================================================================
                                    Variables 0    R² Train     R² Test
--------------------------------------------------------------------------------
0.001      2.1135       -1.3325      2              0.8619       0.8479
0.010      2.1042       -1.3229      2              0.8619       0.8473
0.100      2.0112       -1.2265      2              0.8589       0.8383
0.500      1.5982       -0.7981      2              0.7858       0.7416
1.000      1.0819       -0.2626      2              0.5573       0.4907
2.000      0.0005       -0.0000      1              0.0003       -0.0005
5.000      0.0000       -0.0000      0              0.0000       -0.0007
```

4.3 Comparación Ridge vs Lasso

```python
# Visualización comparativa
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Coeficiente 1
axes[0, 0].semilogx(df_ridge['alpha'], df_ridge['beta1'],
                    'o-', linewidth=2, label='Ridge', markersize=8)
axes[0, 0].semilogx(df_lasso['alpha'], df_lasso['beta1'],
                    's-', linewidth=2, label='Lasso', markersize=8)
axes[0, 0].axhline(y=beta_true[0], color='black', linestyle='--',
                   alpha=0.5, label='Verdadero')
axes[0, 0].set_xlabel(' ')
axes[0, 0].set_ylabel(' ')
axes[0, 0].set_title('Coeficiente   vs  ')
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)

# Coeficiente 2
axes[0, 1].semilogx(df_ridge['alpha'], df_ridge['beta2'],
                    'o-', linewidth=2, label='Ridge', markersize=8)
axes[0, 1].semilogx(df_lasso['alpha'], df_lasso['beta2'],
                    's-', linewidth=2, label='Lasso', markersize=8)
axes[0, 1].axhline(y=beta_true[1], color='black', linestyle='--',
                   alpha=0.5, label='Verdadero')
axes[0, 1].set_xlabel(' ')
axes[0, 1].set_ylabel(' ')
axes[0, 1].set_title('Coeficiente   vs  ')
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

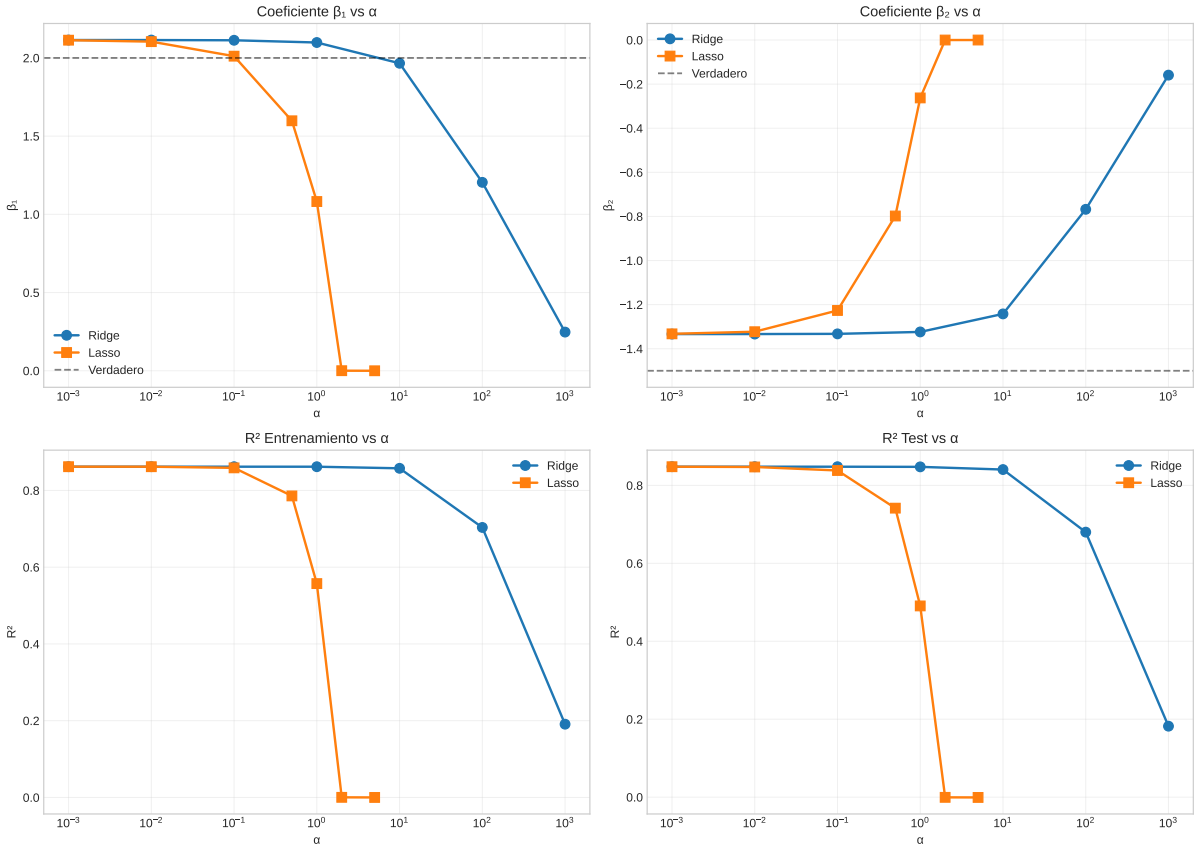
# R² Train
axes[1, 0].semilogx(df_ridge['alpha'], df_ridge['r2_train'],
                    'o-', linewidth=2, label='Ridge', markersize=8)
```

```
axes[1, 0].semilogx(df_lasso['alpha'], df_lasso['r2_train'],
                    's-', linewidth=2, label='Lasso', markersize=8)
axes[1, 0].set_xlabel(' ')
axes[1, 0].set_ylabel('R²')
axes[1, 0].set_title('R² Entrenamiento vs  ')
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3)

# R² Test
axes[1, 1].semilogx(df_ridge['alpha'], df_ridge['r2_test'],
                    'o-', linewidth=2, label='Ridge', markersize=8)
axes[1, 1].semilogx(df_lasso['alpha'], df_lasso['r2_test'],
                    's-', linewidth=2, label='Lasso', markersize=8)
axes[1, 1].set_xlabel(' ')
axes[1, 1].set_ylabel('R²')
axes[1, 1].set_title('R² Test vs  ')
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('ridge_vs_lasso.png', dpi=300, bbox_inches='tight')
print("\nGráfico guardado como 'ridge_vs_lasso.png'")
```

```
Gráfico guardado como 'ridge_vs_lasso.png'
```



5 Elastic Net

5.1 Fundamento matemático

Elastic Net, propuesto por Zou y Hastie (2005), es un método de regularización híbrido que combina las penalizaciones L1 (Lasso) y L2 (Ridge) para aprovechar las ventajas de ambos métodos mientras mitiga sus limitaciones individuales.

5.1.1 Especificación del problema

Elastic Net minimiza la siguiente función objetivo:

$$\min_{\boldsymbol{\beta}} \left\{ \text{RSS}(\boldsymbol{\beta}) + \alpha \left[\rho \|\boldsymbol{\beta}\|_1 + \frac{1-\rho}{2} \|\boldsymbol{\beta}\|_2^2 \right] \right\}$$

donde:

- $\text{RSS}(\boldsymbol{\beta}) = \sum_{i=1}^{n} (Y_i - \mathbf{X}_i^\top \boldsymbol{\beta})^2$ es la suma de residuos al cuadrado
- $\|\boldsymbol{\beta}\|_1 = \sum_{j=1}^{p} |\beta_j|$ es la norma L1 (penalización Lasso)
- $\|\boldsymbol{\beta}\|_2^2 = \sum_{j=1}^{p} \beta_j^2$ es la norma L2 al cuadrado (penalización Ridge)
- $\alpha \geq 0$ es el parámetro que controla la fuerza total de regularización
- $\rho \in [0, 1]$ es el parámetro que controla el balance entre L1 y L2

5.1.2 Forma alternativa de la penalización

La penalización de Elastic Net puede escribirse como:

$$P(\boldsymbol{\beta}) = \alpha\rho \|\boldsymbol{\beta}\|_1 + \alpha \frac{1-\rho}{2} \|\boldsymbol{\beta}\|_2^2$$

que combina:

- **Componente L1**: $\alpha\rho \sum_{j=1}^{p} |\beta_j|$ (promueve sparsity)
- **Componente L2**: $\alpha \frac{1-\rho}{2} \sum_{j=1}^{p} \beta_j^2$ (promueve estabilidad)

5.1.3 Casos especiales

La elección del parámetro ρ determina el comportamiento del modelo:
ρ = 1 (Lasso puro):

$$P(\boldsymbol{\beta}) = \alpha \|\boldsymbol{\beta}\|_1$$

El modelo se reduce a Lasso estándar.
ρ = 0 (Ridge puro):

$$P(\boldsymbol{\beta}) = \frac{\alpha}{2} \|\boldsymbol{\beta}\|_2^2$$

El modelo se reduce a Ridge (con ajuste en la constante de regularización).
0 < ρ < 1 (Elastic Net): Combinación genuina que balancea selección de variables (L1) y estabilidad (L2).

5.1.4 Motivación: limitaciones de Lasso

Elastic Net fue desarrollado para abordar dos limitaciones principales de Lasso:

1. Problema con variables correlacionadas

Cuando dos variables X_j y X_k están altamente correlacionadas:

- Lasso tiende a seleccionar arbitrariamente una y establecer la otra en cero
- Los coeficientes pueden ser muy inestables ante pequeñas perturbaciones en los datos

Solución de Elastic Net: La componente L2 agrupa variables correlacionadas, asignándoles coeficientes similares (como Ridge), mientras que la componente L1 realiza selección de variables.

2. Limitación p > n

Cuando el número de variables excede el número de observaciones ($p > n$):

- Lasso puede seleccionar como máximo n variables
- Esto puede ser problemático si más de n variables son relevantes

Solución de Elastic Net: La componente Ridge elimina esta restricción, permitiendo seleccionar más de n variables si es necesario.

5.1.5 Propiedades del estimador Elastic Net

1. Efecto de agrupamiento (grouping effect)

Supongamos que dos predictores X_j y X_k están perfectamente correlacionados ($X_j = X_k$). Elastic Net tiene la siguiente propiedad:

$$|\hat{\beta}_j - \hat{\beta}_k| \leq \frac{2}{\alpha(1-\rho)} \left|\hat{\beta}_j^{\text{naive}} - \hat{\beta}_k^{\text{naive}}\right|$$

donde $\hat{\beta}^{\text{naive}}$ es el estimador Elastic Net sin la componente L2. Esto muestra que Elastic Net agrupa los coeficientes de variables correlacionadas.

2. Selección de variables

Como Lasso, Elastic Net produce soluciones sparse:

$$\hat{\beta}_j^{\text{EN}} = 0 \quad \text{para algunos } j$$

Pero con más estabilidad que Lasso cuando hay correlaciones.

3. Trade-off sesgo-varianza

Elastic Net permite un control más fino del trade-off:

- α controla la regularización total (sesgo vs varianza global)
- ρ controla el tipo de regularización (selección vs agrupamiento)

5.1.6 Algoritmo de optimización

La solución de Elastic Net no tiene forma cerrada y requiere optimización numérica. El algoritmo más común es **coordinate descent con soft-thresholding**:

Para cada coeficiente β_j, el paso de actualización es:

$$\hat{\beta}_j = \frac{\mathcal{S}_{\alpha\rho}(\tilde{\beta}_j)}{1 + \alpha(1-\rho)}$$

donde:

- $\mathcal{S}_\lambda(\beta) = \text{sign}(\beta)\max(0, |\beta| - \lambda)$ es el operador soft-thresholding
- $\tilde{\beta}_j$ es una cantidad que depende de los residuos parciales

Este algoritmo converge rápidamente en la práctica.

5.1.7 Interpretación geométrica

La región de restricción de Elastic Net es una combinación convexa de:

- El hiperrombo de Lasso (que tiene esquinas en los ejes)
- La hiperesfera de Ridge (que es suave)

Esto crea una región que:

- Tiene "esquinas suavizadas", permitiendo selección de variables
- Es más convexa que el hiperrombo puro, dando más estabilidad

5.1.8 Ventajas de Elastic Net

1. **Balance óptimo**: Combina lo mejor de Lasso (selección) y Ridge (estabilidad).

2. **Manejo de correlaciones**: A diferencia de Lasso, no selecciona arbitrariamente entre variables correlacionadas, sino que las incluye o excluye como grupo.

3. **Supera limitación p > n**: Puede seleccionar más de n variables cuando es apropiado.

4. **Estabilidad mejorada**: Menos sensible a pequeños cambios en los datos que Lasso puro.

5. **Flexibilidad**: Dos parámetros (α y ρ) permiten ajuste más fino del modelo.

6. **Hereda propiedades**: Mantiene propiedades deseables tanto de Ridge como de Lasso.

5.1.9 Limitaciones de Elastic Net

1. **Dos hiperparámetros**: Requiere seleccionar tanto α como ρ, incrementando la complejidad computacional de la validación cruzada.

2. **Interpretación más compleja**: La penalización híbrida es menos intuitiva que L1 o L2 puras.

3. **Costo computacional**: Más costoso que Ridge o Lasso individualmente.

4. **Elección de ρ**: No siempre es claro qué valor de ρ es óptimo; en la práctica se suele usar $\rho = 0.5$ como punto de partida.

5.1.10 Selección de hiperparámetros

Para Elastic Net, se deben seleccionar dos parámetros: α y ρ.
Estrategia 1: Grid search bidimensional

$$(\alpha^*, \rho^*) = \arg\min_{(\alpha,\rho)} \frac{1}{K} \sum_{k=1}^{K} \text{MSE}_k(\alpha, \rho)$$

Se evalúa una grilla de valores (α, ρ) mediante validación cruzada.
Estrategia 2: Grid search con ρ fijo
Frecuentemente se fija $\rho = 0.5$ (balance igual entre L1 y L2) y solo se optimiza α:

$$\alpha^* = \arg\min_{\alpha} \frac{1}{K} \sum_{k=1}^{K} \text{MSE}_k(\alpha, \rho = 0.5)$$

Estrategia 3: Búsqueda secuencial

1. Primero optimizar ρ con un valor fijo de α
2. Luego optimizar α con el ρ óptimo encontrado

5.1.11 Comparación de los tres métodos

Método	Penalización	Selección	Variables correlacionadas	p > n
Ridge	$\alpha \sum \beta_j^2$	No	Coef. similares	OK
Lasso	$\alpha \sum \lVert\beta_j\rVert$	Sí	Selecciona una	Max n variables
Elastic Net	$\alpha[\rho \sum \lVert\beta_j\rVert + \frac{1-\rho}{2} \sum \beta_j^2]$	Sí	Agrupa	OK

5.1.12 Aplicaciones en economía

Elastic Net es particularmente útil en:

1. **Modelos macroeconómicos**: Con múltiples indicadores correlacionados donde se desea tanto selección como estabilidad.
2. **Análisis de panel con muchas variables**: Especialmente cuando hay variables correlacionadas por construcción (ej: rezagos, efectos fijos).
3. **Predicción de series de tiempo**: Con múltiples rezagos y variables explicativas correlacionadas.
4. **Modelos de pricing con características correlacionadas**: En finanzas o valoración inmobiliaria.
5. **High-dimensional econometrics**: Cuando p es comparable o mayor que n y hay correlaciones entre predictores.
6. **Genomic econometrics**: Estudios de economía de la salud con datos genéticos (muchas variables, alta correlación).
7. **Text analysis**: En economía aplicada con muchas variables de texto correlacionadas.

5.1.13 Recomendaciones prácticas

1. **Estandarización**: Siempre estandarizar variables antes de aplicar Elastic Net.
2. **Valor inicial de ρ**: Comenzar con $\rho = 0.5$ como valor por defecto.
3. **Cuando usar cada método**:
   - Use **Ridge** si todas las variables son relevantes y hay multicolinealidad
   - Use **Lasso** si cree que solo pocas variables son relevantes
   - Use **Elastic Net** si hay correlaciones y necesita selección
4. **Validación cruzada**: Use CV tanto para α como para ρ si los recursos computacionales lo permiten.
5. **Estabilidad**: Si los coeficientes de Lasso varían mucho entre folds de CV, considere Elastic Net.

5.1.14 Extensiones y variantes

Sparse Group Lasso: Combina selección de variables individuales (Lasso) con selección de grupos de variables:

$$P(\boldsymbol{\beta}) = \alpha_1 \|\boldsymbol{\beta}\|_1 + \alpha_2 \sum_{g=1}^{G} \|\boldsymbol{\beta}_g\|_2$$

Adaptive Elastic Net: Usa pesos adaptativos para mejorar propiedades de selección consistente:

$$P(\boldsymbol{\beta}) = \alpha \left[\rho \sum_{j=1}^{p} w_j |\beta_j| + \frac{1-\rho}{2} \sum_{j=1}^{p} v_j \beta_j^2 \right]$$

donde los pesos se derivan de estimaciones preliminares.

5.2 Implementación

```
# Entrenar Elastic Net
modelo_elastic = ElasticNet(alpha=0.1, l1_ratio=0.5, max_iter=10000)
modelo_elastic.fit(X_train, y_train)

print("\nELASTIC NET")
print("=" * 70)
print(f"Intercepto: {modelo_elastic.intercept_:.4f}")
print(f"Coeficientes:")
print(f"    : {modelo_elastic.coef_[0]:.4f}")
print(f"    : {modelo_elastic.coef_[1]:.4f}")

r2_elastic = modelo_elastic.score(X_test, y_test)
print(f"\nR² Test: {r2_elastic:.4f}")
```

```
ELASTIC NET
======================================================================
Intercepto: 2.8637
Coeficientes:
   : 1.9592
   : -1.2174

R² Test: 0.8375
```

6 Regresión polinomial

6.1 Fundamento matemático

La regresión polinomial es una extensión de la regresión lineal que permite modelar relaciones no lineales entre la variable dependiente y las variables independientes mediante el uso de términos polinomiales. A pesar de involucrar potencias de las variables, el modelo sigue siendo lineal en los parámetros, lo que permite usar el método de mínimos cuadrados ordinarios.

6.1.1 Especificación del modelo univariado

Para una única variable independiente X, un polinomio de grado d se especifica como:

$$Y_i = \beta_0 + \beta_1 X_i + \beta_2 X_i^2 + \beta_3 X_i^3 + \cdots + \beta_d X_i^d + \varepsilon_i$$

Casos específicos:
Grado 1 (Lineal):

$$Y_i = \beta_0 + \beta_1 X_i + \varepsilon_i$$

Grado 2 (Cuadrático):

$$Y_i = \beta_0 + \beta_1 X_i + \beta_2 X_i^2 + \varepsilon_i$$

Grado 3 (Cúbico):

$$Y_i = \beta_0 + \beta_1 X_i + \beta_2 X_i^2 + \beta_3 X_i^3 + \varepsilon_i$$

6.1.2 Transformación de características

La regresión polinomial transforma el problema en uno de regresión lineal múltiple mediante la creación de nuevas características:
Características originales:

$$\mathbf{X} = [X_1, X_2, \ldots, X_n]^\top$$

Características transformadas:

$$\mathbf{Z} = [1, X, X^2, X^3, \ldots, X^d]$$

donde cada columna es:

$$\mathbf{Z}_j = [1, X_1^j, X_2^j, \ldots, X_n^j]^\top \quad \text{para } j = 0, 1, \ldots, d$$

El modelo entonces es:

$$\mathbf{Y} = \mathbf{Z}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$$

que es lineal en $\boldsymbol{\beta}$ y puede estimarse con OLS:

$$\hat{\boldsymbol{\beta}} = (\mathbf{Z}^\top \mathbf{Z})^{-1} \mathbf{Z}^\top \mathbf{Y}$$

6.1.3 Regresión polinomial multivariada

Para p variables independientes, el modelo puede incluir:
Términos principales:

$$\beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p$$

Términos cuadráticos:

$$\beta_{11} X_1^2 + \beta_{22} X_2^2 + \cdots + \beta_{pp} X_p^2$$

Términos de interacción (productos cruzados):

$$\beta_{12} X_1 X_2 + \beta_{13} X_1 X_3 + \cdots$$

Términos de orden superior:

$$\beta_{111} X_1^3 + \beta_{112} X_1^2 X_2 + \cdots$$

Para un modelo de grado 2 con 2 variables:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_{11} X_1^2 + \beta_{22} X_2^2 + \beta_{12} X_1 X_2 + \varepsilon$$

6.1.4 Número de términos en el modelo

El número total de términos (incluyendo intercepto) en un modelo polinomial de grado d con p variables es:

$$\binom{p+d}{d} = \frac{(p+d)!}{p! \cdot d!}$$

Ejemplos:

- $p = 2$, $d = 2$: $\binom{4}{2} = 6$ términos
- $p = 3$, $d = 2$: $\binom{5}{2} = 10$ términos
- $p = 5$, $d = 3$: $\binom{8}{3} = 56$ términos

Este crecimiento exponencial implica que modelos polinomiales de alto grado con múltiples variables rápidamente se vuelven muy complejos.

6.1.5 Propiedades matemáticas

1. Linealidad en parámetros
Aunque el modelo es no lineal en X, es lineal en $\boldsymbol{\beta}$:

$$Y = \mathbf{Z}\boldsymbol{\beta} + \varepsilon$$

Esto permite usar toda la teoría de regresión lineal.
2. Flexibilidad
Según el **Teorema de aproximación de Weierstrass**, cualquier función continua en un intervalo cerrado puede ser aproximada arbitrariamente bien por un polinomio de grado suficientemente alto.
3. Extrapolación peligrosa
Los polinomios de alto grado pueden comportarse erráticamente fuera del rango de los datos de entrenamiento. La extrapolación es particularmente problemática.

6.1.6 Interpretación de coeficientes

En un modelo cuadrático:

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \varepsilon$$

Efecto marginal:

$$\frac{\partial Y}{\partial X} = \beta_1 + 2\beta_2 X$$

El efecto de X sobre Y depende del valor de X:

- No es constante (como en modelo lineal)
- Varía según el punto de evaluación
- Puede cambiar de signo si β_1 y β_2 tienen signos opuestos

Punto de inflexión (donde cambia la curvatura):
Para un cúbico $Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3$:

$$\frac{\partial^2 Y}{\partial X^2} = 2\beta_2 + 6\beta_3 X = 0 \implies X^* = -\frac{\beta_2}{3\beta_3}$$

Valor extremo (máximo o mínimo):
Para un cuadrático:

$$\frac{\partial Y}{\partial X} = \beta_1 + 2\beta_2 X = 0 \implies X^* = -\frac{\beta_1}{2\beta_2}$$

Es máximo si $\beta_2 < 0$, mínimo si $\beta_2 > 0$.

6.1.7 Problemas de multicolinealidad

Los términos polinomiales están altamente correlacionados entre sí. Por ejemplo, X y X^2 típicamente tienen correlación cercana a 1 cuando $X > 0$.

Consecuencias:

- Matriz $\mathbf{Z}^\top \mathbf{Z}$ cercana a singular
- Coeficientes inestables
- Errores estándar inflados
- Dificultad para invertir la matriz

Solución: Ortogonalización
Usar **polinomios ortogonales** (ej: polinomios de Chebyshev, Legendre):

$$P_0(X) = 1, \quad P_1(X) = X, \quad P_k(X) = \text{combinación ortogonal}$$

satisfaciendo:

$$\sum_{i=1}^{n} P_j(X_i) P_k(X_i) = 0 \quad \text{para } j \neq k$$

6.1.8 Sesgo y varianza con el grado del polinomio

Grado bajo (d pequeño):

- **Sesgo alto**: El modelo es demasiado simple (underfitting)
- **Varianza baja**: Pocas variables, estimaciones estables
- Pobre ajuste tanto en entrenamiento como en prueba

Grado moderado (d óptimo):

- **Balance sesgo-varianza**: Captura la verdadera relación
- Buen ajuste en entrenamiento y prueba

Grado alto (d grande):

- **Sesgo bajo**: El modelo se ajusta muy bien a los datos de entrenamiento
- **Varianza alta**: Muchas variables, sensible a ruido (overfitting)
- Excelente ajuste en entrenamiento, pobre en prueba

6.1.9 Selección del grado óptimo

Criterios de información:
AIC (Akaike Information Criterion):

$$\text{AIC} = n \log(\text{RSS}/n) + 2(d+1)$$

BIC (Bayesian Information Criterion):

$$\text{BIC} = n \log(\text{RSS}/n) + \log(n)(d+1)$$

Seleccionar d que minimiza AIC o BIC.
Validación cruzada:

$$d^* = \arg\min_d \frac{1}{K} \sum_{k=1}^{K} \text{MSE}_k(d)$$

Penalización:
Combinar regresión polinomial con Ridge o Lasso:

$$\min_{\boldsymbol{\beta}} \{\text{RSS}(\boldsymbol{\beta}) + \alpha P(\boldsymbol{\beta})\}$$

donde P es penalización L1, L2, o Elastic Net.

6.1.10 Ventajas de la regresión polinomial

1. **Captura no linealidades**: Modela relaciones curvas naturalmente.
2. **Mantiene linealidad en parámetros**: Permite usar OLS y toda la inferencia asociada.
3. **Flexibilidad**: Puede aproximar muchas formas funcionales.
4. **Implementación simple**: Fácil de implementar transformando características.
5. **Interpretación económica**: Muchas relaciones económicas son naturalmente curvas (rendimientos decrecientes, costos en forma de U).

6.1.11 Limitaciones de la regresión polinomial

1. **Overfitting con grado alto**: Modelos de alto grado se ajustan al ruido.
2. **Extrapolación peligrosa**: Comportamiento errático fuera del rango de datos.
3. **Oscilaciones**: Polinomios de alto grado pueden oscilar wildly (fenómeno de Runge).
4. **Multicolinealidad**: Términos polinomiales altamente correlacionados.
5. **Crecimiento del número de parámetros**: Rápidamente se vuelve intratable con múltiples variables.
6. **Interpretación compleja**: Difícil interpretar coeficientes individuales.
7. **Sensibilidad a outliers**: Especialmente en los extremos del rango.

6.1.12 Aplicaciones en economía

La regresión polinomial es natural para modelar:
1. Funciones de costo:

$$C(Q) = c_0 + c_1 Q + c_2 Q^2 + c_3 Q^3$$

Captura economías y deseconomías de escala.
2. Curvas de Engel:

$$\text{Gasto}_i = \beta_0 + \beta_1 \text{Ingreso}_i + \beta_2 \text{Ingreso}_i^2 + \varepsilon_i$$

Relación no lineal entre ingreso y gasto en diferentes bienes.
3. Funciones de producción con rendimientos variables:

$$Y = \beta_0 + \beta_1 K + \beta_2 K^2 + \beta_3 L + \beta_4 L^2 + \beta_5 KL + \varepsilon$$

4. Curvas de oferta y demanda no lineales:

$$Q_d = \alpha_0 + \alpha_1 P + \alpha_2 P^2 + \varepsilon$$

5. Ciclos económicos: Capturar patrones cíclicos en variables macroeconómicas.
6. Curvas de aprendizaje:

$$\text{Costo}_t = c_0 + c_1 t + c_2 t^2$$

Modelar cómo los costos disminuyen con la experiencia.
7. Relaciones edad-ingreso:

$$\text{Ingreso}_i = \beta_0 + \beta_1 \text{Edad}_i + \beta_2 \text{Edad}_i^2 + \varepsilon_i$$

Típicamente muestra una forma de U invertida (ecuación de Mincer extendida).

6.1.13 Alternativas modernas

Cuando se necesita flexibilidad pero se quiere evitar limitaciones de polinomios globales:

1. Splines:

- Polinomios por partes con restricciones de suavidad
- Más estables que polinomios globales
- Menos propensos a oscilaciones

2. Generalized Additive Models (GAMs):

$$Y = \beta_0 + f_1(X_1) + f_2(X_2) + \cdots + f_p(X_p) + \varepsilon$$

donde f_j son funciones suaves no paramétricas.

3. Redes neuronales:

- Aproximadores universales de funciones
- Más flexibles pero menos interpretables

4. Regression trees:

- Capturan no linealidades mediante particiones
- Interpretables y robustos

6.1.14 Recomendaciones prácticas

1. **Comenzar simple**: Probar modelo lineal primero, luego cuadrático, luego cúbico.
2. **No exceder grado 4-5**: En la práctica, grados mayores rara vez mejoran y causan problemas.
3. **Estandarizar**: Centrar y escalar X antes de crear potencias.
4. **Usar penalización**: Combinar con Ridge/Lasso si d es alto.
5. **Validación cruzada**: Usar CV para seleccionar d óptimo.
6. **Verificar extrapolación**: Ser muy cauteloso al predecir fuera del rango de entrenamiento.
7. **Considerar splines**: Si necesita grados altos, considere splines en su lugar.
8. **Graficar siempre**: Visualizar la curva ajustada para detectar comportamientos extraños.

6.2 Implementación

```
# Generar datos con relación no lineal
np.random.seed(42)
X_poly_1d = np.random.uniform(-3, 3, 100)
Y_poly = 0.5 * X_poly_1d**2 - 2 * X_poly_1d + 1 + np.random.randn(100) * 2

# Ordenar para visualización
```

```
sort_idx = np.argsort(X_poly_1d)
X_poly_1d = X_poly_1d[sort_idx]
Y_poly = Y_poly[sort_idx]

X_poly_1d = X_poly_1d.reshape(-1, 1)

# Dividir datos
X_train_poly, X_test_poly, y_train_poly, y_test_poly = train_test_split(
    X_poly_1d, Y_poly, test_size=0.3, random_state=42
)

# Probar diferentes grados
grados = [1, 2, 3, 5, 8]
colores = ['blue', 'green', 'red', 'purple', 'orange']

plt.figure(figsize=(14, 8))

print("\nREGRESIÓN POLINOMIAL")
print("=" * 70)
print(f"{'Grado':<8} {'R² Train':<12} {'R² Test':<12} {'MSE Train':<12} {'MSE Test':<
print("-" * 70)

for grado, color in zip(grados, colores):
    # Crear características polinomiales
    poly_features = PolynomialFeatures(degree=grado, include_bias=False)
    X_train_poly_transformed = poly_features.fit_transform(X_train_poly)
    X_test_poly_transformed = poly_features.transform(X_test_poly)

    # Entrenar modelo
    modelo = LinearRegression()
    modelo.fit(X_train_poly_transformed, y_train_poly)

    # Evaluar
    r2_train = modelo.score(X_train_poly_transformed, y_train_poly)
    r2_test = modelo.score(X_test_poly_transformed, y_test_poly)

    y_pred_train = modelo.predict(X_train_poly_transformed)
    y_pred_test = modelo.predict(X_test_poly_transformed)

    mse_train = mean_squared_error(y_train_poly, y_pred_train)
    mse_test = mean_squared_error(y_test_poly, y_pred_test)

    print(f"{grado:<8} {r2_train:<12.4f} {r2_test:<12.4f} "
          f"{mse_train:<12.4f} {mse_test:<12.4f}")

    # Graficar
    X_plot = np.linspace(X_poly_1d.min(), X_poly_1d.max(), 300).reshape(-1, 1)
```

```
    X_plot_transformed = poly_features.transform(X_plot)
    y_plot = modelo.predict(X_plot_transformed)

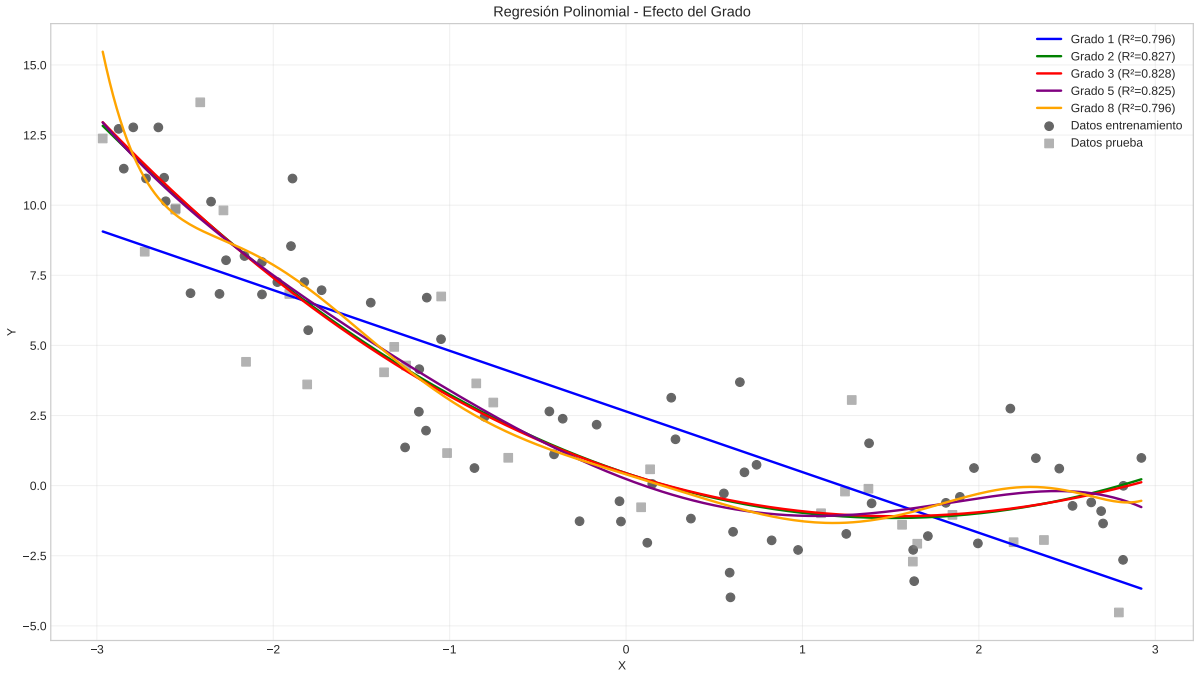
    plt.plot(X_plot, y_plot, color=color, linewidth=2,
             label=f'Grado {grado} (R²={r2_test:.3f})')

# Datos originales
plt.scatter(X_train_poly, y_train_poly, alpha=0.6, s=50,
           color='black', label='Datos entrenamiento')
plt.scatter(X_test_poly, y_test_poly, alpha=0.6, s=50,
           marker='s', color='gray', label='Datos prueba')

plt.xlabel('X')
plt.ylabel('Y')
plt.title('Regresión Polinomial - Efecto del Grado')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('polynomial_regression.png', dpi=300, bbox_inches='tight')
print("\nGráfico guardado como 'polynomial_regression.png'")
```

```
REGRESIÓN POLINOMIAL
======================================================================
Grado    R² Train     R² Test      MSE Train    MSE Test
----------------------------------------------------------------------
1        0.6946       0.7957       6.4986       4.5762
2        0.8668       0.8265       2.8340       3.8863
3        0.8669       0.8276       2.8315       3.8613
5        0.8702       0.8254       2.7615       3.9118
8        0.8739       0.7964       2.6829       4.5608

Gráfico guardado como 'polynomial_regression.png'
```



7 K-Nearest Neighbors Regressor

7.1 Fundamento matemático

K-Nearest Neighbors (KNN) para regresión es un método no paramétrico que realiza predicciones basándose en el promedio de los valores de las observaciones más cercanas en el espacio de características. A diferencia de los métodos paramétricos como OLS o Ridge, KNN no asume una forma funcional específica para la relación entre variables.

7.1.1 Definición formal

Dado un conjunto de entrenamiento $\mathcal{D} = \{(\mathbf{X}_i, Y_i)\}_{i=1}^{n}$ y un nuevo punto $\mathbf{X}_0$, el estimador KNN predice:

$$\hat{Y}_0 = \frac{1}{k} \sum_{i \in \mathcal{N}_k(\mathbf{X}_0)} Y_i$$

donde:

- k es el número de vecinos a considerar
- $\mathcal{N}_k(\mathbf{X}_0)$ es el conjunto de los k vecinos más cercanos a $\mathbf{X}_0$
- La cercanía se mide típicamente mediante una métrica de distancia

Interpretación: La predicción es simplemente el promedio de las k observaciones más similares al nuevo punto.

7.1.2 Métricas de distancia

La elección de la métrica de distancia es crucial en KNN. Las más comunes son:
1. Distancia Euclidiana (la más utilizada):

$$d(\mathbf{X}_i, \mathbf{X}_j) = \sqrt{\sum_{m=1}^{p} (X_{im} - X_{jm})^2} = \|\mathbf{X}_i - \mathbf{X}_j\|_2$$

2. Distancia Manhattan (también llamada L1 o city-block):

$$d(\mathbf{X}_i, \mathbf{X}_j) = \sum_{m=1}^{p} |X_{im} - X_{jm}| = \|\mathbf{X}_i - \mathbf{X}_j\|_1$$

3. Distancia Minkowski (generalización):

$$d(\mathbf{X}_i, \mathbf{X}_j) = \left(\sum_{m=1}^{p} |X_{im} - X_{jm}|^q \right)^{1/q} = \|\mathbf{X}_i - \mathbf{X}_j\|_q$$

donde:

- $q = 1$: Distancia Manhattan
- $q = 2$: Distancia Euclidiana
- $q \to \infty$: Distancia de Chebyshev (máximo)

4. Distancia de Chebyshev:

$$d(\mathbf{X}_i, \mathbf{X}_j) = \max_{m=1,\ldots,p} |X_{im} - X_{jm}|$$

5. Distancia de Mahalanobis (considera correlaciones):

$$d(\mathbf{X}_i, \mathbf{X}_j) = \sqrt{(\mathbf{X}_i - \mathbf{X}_j)^\top \mathbf{\Sigma}^{-1} (\mathbf{X}_i - \mathbf{X}_j)}$$

donde $\mathbf{\Sigma}$ es la matriz de covarianza de las características.

7.1.3 Algoritmo KNN

Paso 1: Dado un nuevo punto $\mathbf{X}_0$, calcular su distancia a todos los puntos de entrenamiento:

$$d_i = d(\mathbf{X}_0, \mathbf{X}_i) \quad \text{para } i = 1, \ldots, n$$

Paso 2: Ordenar las distancias en orden ascendente:

$$d_{(1)} \leq d_{(2)} \leq \cdots \leq d_{(n)}$$

Paso 3: Seleccionar los k puntos con menores distancias.
Paso 4: Calcular la predicción como el promedio de sus valores:

$$\hat{Y}_0 = \frac{1}{k} \sum_{i=1}^{k} Y_{(i)}$$

donde $Y_{(i)}$ es el valor de Y correspondiente a la i-ésima menor distancia.

7.1.4 Variantes ponderadas

En lugar de un promedio simple, se puede usar un **promedio ponderado** donde observaciones más cercanas tienen más peso:

$$\hat{Y}_0 = \frac{\sum_{i=1}^{k} w_i Y_{(i)}}{\sum_{i=1}^{k} w_i}$$

Pesos comunes:
1. Uniforme (estándar):

$$w_i = 1 \quad \text{para todo } i$$

2. Inverso de la distancia:

$$w_i = \frac{1}{d_i + \epsilon}$$

donde $\epsilon > 0$ es un término pequeño para evitar división por cero.
3. Kernel gaussiano:

$$w_i = \exp\left(-\frac{d_i^2}{2\sigma^2}\right)$$

4. Kernel de Epanechnikov:

$$w_i = \max\left(0, 1 - \frac{d_i^2}{h^2}\right)$$

7.1.5 Propiedades estadísticas

1. Consistencia
Bajo condiciones regulares, KNN es consistente:

$$\lim_{n\to\infty} E[(\hat{Y}_0 - Y_0)^2] = 0$$

siempre que $k \to \infty$ y $k/n \to 0$ cuando $n \to \infty$.
2. Sesgo y varianza
Para un punto fijo $\mathbf{X}_0$:
Sesgo:

$$\text{Bias}[\hat{Y}_0] = E[\hat{Y}_0] - E[Y_0|\mathbf{X}_0]$$

El sesgo disminuye cuando k aumenta (se promedian más observaciones cercanas).
Varianza:

$$\text{Var}[\hat{Y}_0] = \text{Var}\left[\frac{1}{k}\sum_{i\in\mathcal{N}_k} Y_i\right] \approx \frac{\sigma^2}{k}$$

La varianza disminuye con k más grande.
3. Trade-off con k

- **k pequeño**:
  - Baja bias (modelo flexible, se adapta a variaciones locales)
  - Alta varianza (sensible a ruido, predicciones inestables)
  - Riesgo de overfitting

- **k grande**:

  - Alta bias (modelo menos flexible, suaviza demasiado)
  - Baja varianza (predicciones estables)
  - Riesgo de underfitting

4. Error de predicción esperado

$$E[(Y_0 - \hat{Y}_0)^2] = \text{Bias}^2[\hat{Y}_0] + \text{Var}[\hat{Y}_0] + \sigma^2$$

Existe un k óptimo que minimiza este error.

7.1.6 Selección del parámetro k

Validación cruzada leave-one-out:

$$\text{CV}(k) = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_{-i}(k))^2$$

donde $\hat{Y}_{-i}(k)$ es la predicción KNN para $\mathbf{X}_i$ usando todos los datos excepto la observación i.

Validación cruzada K-fold:

$$k^* = \arg\min_k \frac{1}{K}\sum_{j=1}^{K}\text{MSE}_j(k)$$

Regla práctica:

$$k \approx \sqrt{n}$$

Esta es una heurística común para valores iniciales.

7.1.7 Complejidad computacional

Búsqueda naive:

- **Entrenamiento**: $O(1)$ (solo almacenar datos)
- **Predicción**: $O(np)$ por cada nueva observación

  - Calcular n distancias de dimensión p
  - Encontrar los k mínimos

Estructuras eficientes (KD-trees, Ball trees):

- **Entrenamiento**: $O(np \log n)$
- **Predicción**: $O(\log n)$ en promedio (mejor caso)

Sin embargo, en alta dimensionalidad (p grande), estas estructuras pierden eficiencia ("curse of dimensionality").

7.1.8 Sensibilidad a la escala

KNN es **extremadamente sensible** a la escala de las variables. Una variable con rango [0, 1000] dominará una variable con rango [0, 1] en el cálculo de distancias.

Solución: Estandarización

Antes de aplicar KNN, estandarizar cada característica:

$$\tilde{X}_{im} = \frac{X_{im} - \mu_m}{\sigma_m}$$

donde μ_m y σ_m son la media y desviación estándar de la característica m.

Alternativamente, normalizar al rango [0, 1]:

$$\tilde{X}_{im} = \frac{X_{im} - \min_i X_{im}}{\max_i X_{im} - \min_i X_{im}}$$

7.1.9 Maldición de la dimensionalidad

En alta dimensionalidad, KNN sufre de la "curse of dimensionality":

Problema 1: Distancias se vuelven uniformes

En dimensiones altas, las distancias entre cualquier par de puntos tienden a ser similares:

$$\lim_{p \to \infty} \frac{d_{\max} - d_{\min}}{d_{\min}} \to 0$$

Problema 2: Volumen requerido para capturar k vecinos

Para mantener k vecinos en una región local, el volumen requerido crece exponencialmente con p:

$$V \propto r^p$$

donde r es el radio y p es la dimensión.

Consecuencia: En alta dimensionalidad, los "vecinos cercanos" pueden estar muy lejos, y KNN pierde efectividad.

Regla práctica: KNN funciona mejor cuando $p < 10 - 20$.

7.1.10 Ventajas del método KNN

1. **No paramétrico**: No asume forma funcional específica para la relación.

2. **Simplicidad conceptual**: Muy intuitivo y fácil de entender.

3. **Flexibilidad**: Puede capturar relaciones muy complejas y no lineales.

4. **No requiere entrenamiento**: Solo almacena datos (lazy learning).

5. **Actualización online**: Fácil incorporar nuevos datos.

6. **Robusto a outliers** (con k grande): El promedio reduce influencia de valores extremos.

7.1.11 Limitaciones del método KNN

1. **Computacionalmente costoso en predicción**: Necesita calcular distancias a todos los puntos de entrenamiento.
2. **Sensible a la escala**: Requiere estandarización obligatoria.
3. **Maldición de la dimensionalidad**: Performance deteriora rápidamente con p grande.
4. **Requiere gran memoria**: Debe almacenar todo el dataset de entrenamiento.
5. **Sensible a características irrelevantes**: Variables no informativas contaminan el cálculo de distancias.
6. **Dificultad en selección de k**: No hay regla universal; requiere experimentación.
7. **No proporciona forma funcional**: No hay coeficientes interpretables.
8. **Límites de decisión irregulares**: Pueden ser demasiado complejos.

7.1.12 Aplicaciones en economía

KNN es útil en contextos económicos donde:
1. Predicción de precios basada en comparables:

- **Valuación inmobiliaria**: Estimar precio de una casa basándose en casas similares (ubicación, tamaño, características).
- **Pricing de activos**: Valorar activos basándose en activos similares.

2. Recomendación y matching:

- **Sistemas de recomendación**: Sugerir productos basados en preferencias de usuarios similares.
- **Mercados laborales**: Emparejar trabajadores con empleos similares a sus características.

3. Forecasting no paramétrico:

- **Series de tiempo**: Predecir valores futuros basándose en patrones históricos similares.
- **Análisis de ciclos**: Identificar períodos históricos similares al actual.

4. Imputación de valores faltantes:

- Imputar datos económicos faltantes usando información de entidades similares.

5. Detección de anomalías:

- **Fraude**: Identificar transacciones muy diferentes de las "normales".
- **Outliers**: Detectar observaciones atípicas en datos económicos.

6. Clustering y segmentación:

- Agrupar empresas, países, o individuos con características similares.

7.1.13 Comparación con métodos paramétricos

Aspecto	KNN	Métodos Paramétricos (OLS, Ridge, Lasso)
Supuestos	Mínimos	Forma funcional específica
Interpretabilidad	Baja (no hay coeficientes)	Alta (coeficientes interpretables)
Flexibilidad	Alta	Media-Baja
Extrapolación	Problemática	Más confiable
Dimensionalidad	Sufre con p alto	Maneja mejor (especialmente con regularización)
Costo computacional	Alto en predicción	Bajo en predicción
Datos necesarios	Más datos generalmente mejor	Puede funcionar con menos datos

7.1.14 Extensiones y variantes

1. Locally Weighted Regression (LOESS):
Combina KNN con regresión:

$$\hat{Y}_0 = \arg\min_{\beta_0, \boldsymbol{\beta}} \sum_{i \in \mathcal{N}_k} w_i (Y_i - \beta_0 - \mathbf{X}_i^\top \boldsymbol{\beta})^2$$

Ajusta regresión local en vecindad de cada punto.
2. Radius Neighbors Regression:
En lugar de k fijo, incluye todos los puntos dentro de un radio r:

$$\hat{Y}_0 = \frac{1}{|\mathcal{N}_r|} \sum_{i \in \mathcal{N}_r(\mathbf{X}_0)} Y_i$$

donde $\mathcal{N}_r(\mathbf{X}_0) = \{i : d(\mathbf{X}_0, \mathbf{X}_i) \leq r\}$.
3. Feature-Weighted KNN:
Asigna pesos diferentes a diferentes características:

$$d(\mathbf{X}_i, \mathbf{X}_j) = \sqrt{\sum_{m=1}^{p} w_m (X_{im} - X_{jm})^2}$$

Los pesos se pueden aprender para mejorar performance.

7.1.15 Recomendaciones prácticas

1. **Siempre estandarizar**: Crucial para resultados válidos.
2. **Seleccionar k por CV**: No confiar en reglas de oro; validar empíricamente.
3. **Probar múltiples métricas**: Euclidiana es estándar, pero otras pueden funcionar mejor.
4. **Reducir dimensionalidad**: Considerar PCA o selección de features si p es grande.
5. **Comparar con métodos paramétricos**: KNN no siempre es superior; comparar performance.
6. **Considerar computational cost**: Para datasets grandes, puede ser prohibitivamente lento.
7. **Usar para benchmarking**: KNN es útil como baseline no paramétrico.
8. **Visualizar vecindarios**: En 2D-3D, visualizar qué puntos son considerados "vecinos".

7.2 Implementación

```
# Estandarizar datos (importante para KNN)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Probar diferentes valores de k
k_values = [1, 3, 5, 10, 20, 30, 50]

print("\nK-NEAREST NEIGHBORS REGRESSOR")
print("=" * 70)
print(f"{'k':<8} {'R² Train':<12} {'R² Test':<12} {'MSE Test':<12}")
print("-" * 70)

resultados_knn = []

for k in k_values:
    # Entrenar modelo
    modelo_knn = KNeighborsRegressor(n_neighbors=k)
    modelo_knn.fit(X_train_scaled, y_train)

    # Evaluar
    r2_train = modelo_knn.score(X_train_scaled, y_train)
    r2_test = modelo_knn.score(X_test_scaled, y_test)

    y_pred_test = modelo_knn.predict(X_test_scaled)
    mse_test = mean_squared_error(y_test, y_pred_test)

    resultados_knn.append({
```

```
        'k': k,
        'r2_train': r2_train,
        'r2_test': r2_test,
        'mse_test': mse_test
    })

    print(f"{k:<8} {r2_train:<12.4f} {r2_test:<12.4f} {mse_test:<12.4f}")

# Visualizar
df_knn = pd.DataFrame(resultados_knn)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

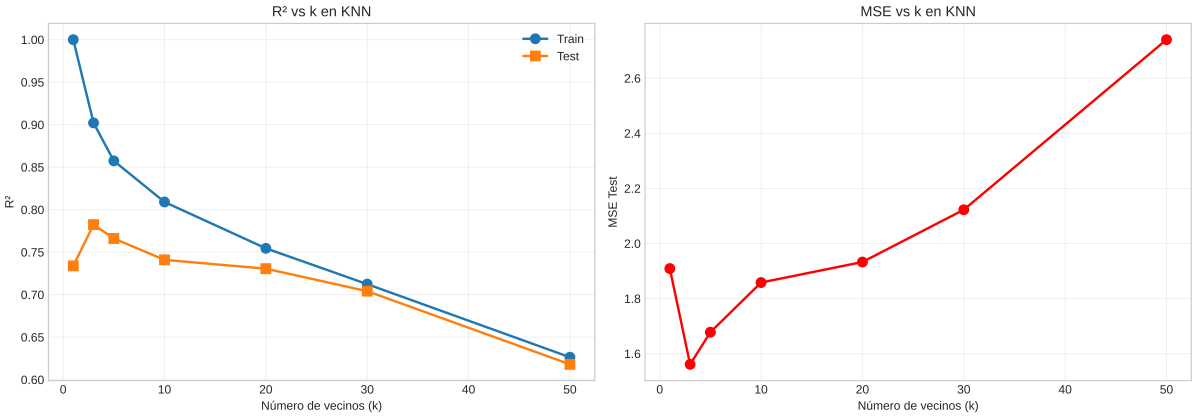
axes[0].plot(df_knn['k'], df_knn['r2_train'],
             'o-', linewidth=2, label='Train', markersize=8)
axes[0].plot(df_knn['k'], df_knn['r2_test'],
             's-', linewidth=2, label='Test', markersize=8)
axes[0].set_xlabel('Número de vecinos (k)')
axes[0].set_ylabel('R²')
axes[0].set_title('R² vs k en KNN')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

axes[1].plot(df_knn['k'], df_knn['mse_test'],
             'o-', linewidth=2, color='red', markersize=8)
axes[1].set_xlabel('Número de vecinos (k)')
axes[1].set_ylabel('MSE Test')
axes[1].set_title('MSE vs k en KNN')
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('knn_regression.png', dpi=300, bbox_inches='tight')
print("\nGráfico guardado como 'knn_regression.png'")
```

```
K-NEAREST NEIGHBORS REGRESSOR
======================================================================
k        R² Train     R² Test      MSE Test
----------------------------------------------------------------------
1        1.0000       0.7339       1.9089
3        0.9021       0.7823       1.5612
5        0.8574       0.7661       1.6778
10       0.8090       0.7409       1.8582
20       0.7545       0.7305       1.9327
30       0.7124       0.7041       2.1224
50       0.6264       0.6180       2.7400

Gráfico guardado como 'knn_regression.png'
```



8 Métricas de evaluación

8.1 Fundamento matemático

La evaluación de modelos de regresión es crucial para comparar diferentes algoritmos y seleccionar el modelo óptimo. A diferencia de la clasificación (que usa accuracy, precision, recall), la regresión requiere métricas que cuantifiquen la magnitud del error de predicción.

8.1.1 Notación

Para un conjunto de n observaciones:

- Y_i: Valor real de la variable dependiente para la observación i
- $\hat{Y}_i$: Valor predicho por el modelo
- $\bar{Y} = \frac{1}{n}\sum_{i=1}^{n} Y_i$: Media de los valores reales

8.1.2 Mean Squared Error (MSE)

Definición:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$$

Interpretación:

- Promedio de los errores al cuadrado
- Penaliza más los errores grandes (debido al cuadrado)
- Unidades: cuadrado de las unidades de Y
- Rango: $[0, \infty)$, donde 0 indica predicción perfecta

Propiedades:

1. **Diferenciabilidad**: MSE es diferenciable en todas partes, facilitando optimización.
2. **Sensibilidad a outliers**: El cuadrado magnifica errores grandes.
3. **Descomposición**:

$$\text{MSE} = \text{Bias}^2[\hat{Y}] + \text{Var}[\hat{Y}] + \sigma^2$$

donde:

- $\text{Bias}^2[\hat{Y}] = (E[\hat{Y}] - Y)^2$: Error sistemático

- $\text{Var}[\hat{Y}]$: Variabilidad de las predicciones
- σ^2: Varianza irreducible (ruido)

Ventajas:

- Matemáticamente conveniente para optimización
- Conexión directa con likelihood gaussiana
- Penaliza errores grandes

Desventajas:

- Sensible a outliers
- Unidades no son interpretables directamente
- No es robusto

Cuándo usar: Cuando errores grandes son particularmente indeseables y los datos no tienen outliers extremos.

8.1.3 Root Mean Squared Error (RMSE)

Definición:

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2}$$

Interpretación:

- Desviación típica de los errores de predicción
- Mismas unidades que Y (interpretable)
- Representa el "error promedio" en las mismas unidades que la variable objetivo

Propiedades:

1. **Interpretabilidad**: Unidades coinciden con las de Y.
2. **Relación con MSE**: $\text{RMSE} = \sqrt{\text{MSE}}$
3. **Comparación entre modelos**: Solo comparable si modelos se evalúan en mismo conjunto de datos.

Ventajas:

- Unidades interpretables
- Mantiene sensibilidad a errores grandes
- Más interpretable que MSE

Desventajas:

- Aún sensible a outliers
- No acotado superiormente

Cuándo usar: Para comunicar el error en unidades comprensibles (ej: "el modelo predice con un error promedio de 5000 soles").

8.1.4 Mean Absolute Error (MAE)

Definición:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|Y_i - \hat{Y}_i|$$

Interpretación:

- Promedio de los valores absolutos de los errores
- Todos los errores pesan igual (lineal)
- Unidades: mismas que Y

Propiedades:

1. **Robustez**: Menos sensible a outliers que MSE/RMSE.
2. **Linealidad**: Error de magnitud 2ϵ pesa exactamente el doble que error de magnitud ϵ.
3. **No diferenciable en cero**: Complica optimización con métodos basados en gradientes.
4. **Relación con mediana**: MAE es minimizado cuando $\hat{Y}_i = \text{mediana}(Y)$.

Ventajas:

- Robusto a outliers
- Interpretación intuitiva
- Unidades interpretables

Desventajas:

- No penaliza errores grandes tanto como MSE
- No diferenciable en el origen
- Puede subponderar errores grandes que son importantes

Cuándo usar: Cuando hay outliers o cuando todos los errores deben pesar igual independiente de su magnitud.

8.1.5 Comparación MAE vs RMSE

Para un mismo conjunto de errores:

$$\text{MAE} \leq \text{RMSE}$$

La desigualdad es estricta excepto cuando todos los errores tienen la misma magnitud.
Ratio RMSE/MAE:

- Cercano a 1: Errores uniformemente distribuidos
- Alejado de 1: Algunos errores muy grandes (outliers)

$$\frac{\text{RMSE}}{\text{MAE}} \geq 1$$

8.1.6 Coeficiente de Determinación (R^2)

Definición:

$$R^2 = 1 - \frac{\text{RSS}}{\text{TSS}} = 1 - \frac{\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2}{\sum_{i=1}^{n}(Y_i - \bar{Y})^2}$$

donde:

- $\text{RSS} = \sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$: Residual Sum of Squares (suma de cuadrados residuales)
- $\text{TSS} = \sum_{i=1}^{n}(Y_i - \bar{Y})^2$: Total Sum of Squares (variabilidad total)

Forma alternativa:

$$R^2 = \frac{\text{ESS}}{\text{TSS}} = \frac{\sum_{i=1}^{n}(\hat{Y}_i - \bar{Y})^2}{\sum_{i=1}^{n}(Y_i - \bar{Y})^2}$$

donde ESS es Explained Sum of Squares.
Interpretación:

- Proporción de varianza de Y explicada por el modelo
- $R^2 = 0$: El modelo no explica nada (tan bueno como predecir $\bar{Y}$)
- $R^2 = 1$: El modelo explica perfectamente la varianza
- $R^2 < 0$: El modelo es peor que simplemente predecir la media (posible en conjunto de prueba)

Propiedades:

1. **Rango en training**: $0 \leq R^2 \leq 1$ (con intercepto)
2. **Rango en testing**: $-\infty < R^2 \leq 1$ (puede ser negativo)
3. **Relación con correlación**: En regresión lineal simple:

   $$R^2 = \rho^2_{Y,\hat{Y}}$$

   donde ρ es el coeficiente de correlación de Pearson.
4. **Sensibilidad a variables**: R^2 nunca disminuye al agregar variables (en training).
5. **No tiene unidades**: Es una proporción, interpretable sin conocer unidades de Y.

Ventajas:

- Fácil interpretación (porcentaje de varianza explicada)
- Sin unidades (comparable entre problemas)
- Intuitivo para audiencias no técnicas

Desventajas:

- Puede ser engañoso (alto R^2 no implica buen modelo)
- Siempre aumenta con más variables en training (favorece sobreajuste)
- Puede ser negativo en test set
- No indica si modelo está bien especificado

Cuándo usar: Para reportar qué proporción de variabilidad se captura, especialmente en contextos académicos o cuando se comunica con no especialistas.

8.1.7 R^2 Ajustado

Definición:

$$R^2_{\text{adj}} = 1 - \frac{\text{RSS}/(n-p-1)}{\text{TSS}/(n-1)} = 1 - (1-R^2)\frac{n-1}{n-p-1}$$

donde:

- n: número de observaciones
- p: número de variables independientes (sin contar intercepto)

Interpretación:

- Ajusta R^2 penalizando por número de variables
- Puede disminuir al agregar variables irrelevantes
- Más apropiado para comparar modelos con diferente número de variables

Propiedades:

1. **Penalización por complejidad**:

$$R^2_{\text{adj}} = R^2 - \frac{p}{n-p-1}(1-R^2)$$

2. **Comparación**: $R^2_{\text{adj}} \leq R^2$ siempre.

3. **Puede ser negativo**: Tanto en training como en testing.

Ventajas:

- Penaliza modelos complejos
- Útil para selección de variables
- Más justo para comparar modelos

Desventajas:

- Aún puede favorecer modelos sobreajustados
- Interpretación menos intuitiva que R^2

Cuándo usar: Al comparar modelos con diferente número de variables, especialmente en regresión lineal clásica.

8.1.8 Mean Absolute Percentage Error (MAPE)

Definición:

$$\text{MAPE} = \frac{100\%}{n}\sum_{i=1}^{n}\left|\frac{Y_i - \hat{Y}_i}{Y_i}\right|$$

Interpretación:

- Error promedio en términos porcentuales
- Independiente de la escala
- Expresado como porcentaje

Propiedades:

1. **Scale-independent**: Útil para comparar modelos en diferentes escalas.
2. **Asimétrico**: Penaliza más subestimaciones que sobreestimaciones.
3. **Indefinido para** $Y_i = 0$: No se puede calcular si hay valores cero.

Ventajas:

- Interpretación intuitiva (porcentaje)
- No depende de escala
- Fácil de comunicar

Desventajas:

- Indefinido cuando $Y_i = 0$
- Asimétrico (penaliza diferente sub- y sobre-estimaciones)
- Puede ser inflado por valores pequeños de Y_i

Cuándo usar: Cuando se necesita comparar performance en diferentes escalas o comunicar errores como porcentajes.

8.1.9 Median Absolute Error (MedAE)

Definición:

$$\text{MedAE} = \text{median}(|Y_1 - \hat{Y}_1|, |Y_2 - \hat{Y}_2|, \ldots, |Y_n - \hat{Y}_n|)$$

Interpretación:

- Mediana de los errores absolutos
- Extremadamente robusto a outliers
- 50% de predicciones tienen error menor que MedAE

Ventajas:

- Muy robusto a outliers
- Representativo del error “típico”

Desventajas:

- Ignora magnitud de errores grandes
- Menos usado, menos familiar

Cuándo usar: Cuando hay muchos outliers y se necesita una métrica muy robusta.

8.1.10 Elección de métrica según contexto

Contexto 1: Predicción de precios de viviendas
Usar **RMSE** o **MAE**:

- RMSE: Si errores grandes en casas caras son muy problemáticos
- MAE: Si se quiere error interpretable en soles

Contexto 2: Forecasting de demanda
Usar **MAPE**:

- Errores como porcentaje son más interpretables
- Permite comparar performance en productos de diferentes escalas

Contexto 3: Investigación académica
Usar **R^2** y **RMSE**:

- R^2: Para reportar varianza explicada
- RMSE: Para magnitud del error

Contexto 4: Detección de outliers
Usar **MAE** o **MedAE**:

- Robustas a outliers
- No dominadas por observaciones extremas

8.1.11 Tabla comparativa de métricas

Métrica	Unidades	Rango	Sensibilidad outliers	Interpretación
MSE	Y^2	$[0, \infty)$	Alta	Error cuadrático medio
RMSE	Y	$[0, \infty)$	Alta	Error típico en unidades de Y
MAE	Y	$[0, \infty)$	Media	Error absoluto medio
R^2	Sin unidades	$(-\infty, 1]$	Media	Proporción varianza explicada
R^2	Sin unidades	$(-\infty, 1]$	Media	R^2 ajustado por complejidad
MAPE	Porcentaje	$[0, \infty)$	Baja	Error porcentual medio
MedAE	Y	$[0, \infty)$	Muy baja	Error mediano

8.1.12 Relaciones matemáticas entre métricas

1. MSE, RMSE y varianza del error:

$$\text{MSE} = \text{Var}(\text{error}) + \text{Bias}^2(\text{error})$$

2. R^2 y MSE:

$$R^2 = 1 - \frac{\text{MSE}}{\text{Var}(Y)}$$

3. Desigualdad entre métricas:

$$\text{MedAE} \leq \text{MAE} \leq \text{RMSE} \leq \max_i |Y_i - \hat{Y}_i|$$

8.1.13 Recomendaciones prácticas

1. **Usar múltiples métricas**: No confiar en una sola; diferentes métricas revelan diferentes aspectos.
2. **Considerar el contexto**: La métrica apropiada depende del problema y las consecuencias de los errores.
3. **Reportar en test set**: Métricas en training pueden ser engañosas (overfitting).
4. **Estandarizar comparaciones**: Usar mismos datos de test para comparar modelos.
5. **Documentar claramente**: Especificar qué métrica se optimiza y por qué.
6. **Considerar distribución de errores**: No solo el promedio; visualizar histogramas de errores.
7. **Cross-validation**: Reportar media y desviación estándar de métricas en CV.
8. **Business metrics**: Complementar con métricas específicas del negocio (ej: impacto económico de errores).

8.2 Implementación de métricas

```
def calcular_metricas(y_true, y_pred, modelo_nombre="Modelo"):
    """Calcula todas las métricas de regresión"""

    # Calcular métricas
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)

    # Calcular R² ajustado manualmente
    n = len(y_true)
    p = 2  # número de predictores en nuestro caso
    r2_adj = 1 - (1 - r2) * (n - 1) / (n - p - 1)

    # Imprimir
    print(f"\n{modelo_nombre}")
    print("-" * 50)
    print(f"  MSE:          {mse:.4f}")
    print(f"  RMSE:         {rmse:.4f}")
    print(f"  MAE:          {mae:.4f}")
    print(f"  R²:           {r2:.4f}")
    print(f"  R² Ajustado:  {r2_adj:.4f}")

    return {
        'modelo': modelo_nombre,
        'mse': mse,
```

```
        'rmse': rmse,
        'mae': mae,
        'r2': r2,
        'r2_adj': r2_adj
    }

# Calcular métricas para cada modelo
print("\nMÉTRICAS DE EVALUACIÓN - CONJUNTO DE PRUEBA")
print("=" * 70)

metricas_modelos = []

# OLS
metricas_modelos.append(calcular_metricas(y_test, y_pred_ols_test, "OLS"))

# Ridge (con mejor alpha)
modelo_ridge_final = Ridge(alpha=mejor_alpha)
modelo_ridge_final.fit(X_train, y_train)
y_pred_ridge = modelo_ridge_final.predict(X_test)
metricas_modelos.append(calcular_metricas(y_test, y_pred_ridge, "Ridge"))

# Lasso
mejor_alpha_lasso = df_lasso.loc[df_lasso['r2_test'].idxmax(), 'alpha']
modelo_lasso_final = Lasso(alpha=mejor_alpha_lasso, max_iter=10000)
modelo_lasso_final.fit(X_train, y_train)
y_pred_lasso = modelo_lasso_final.predict(X_test)
metricas_modelos.append(calcular_metricas(y_test, y_pred_lasso, "Lasso"))

# KNN
mejor_k = df_knn.loc[df_knn['r2_test'].idxmax(), 'k']
modelo_knn_final = KNeighborsRegressor(n_neighbors=int(mejor_k))
modelo_knn_final.fit(X_train_scaled, y_train)
y_pred_knn = modelo_knn_final.predict(X_test_scaled)
metricas_modelos.append(calcular_metricas(y_test, y_pred_knn, "KNN"))

# Crear DataFrame comparativo
df_metricas = pd.DataFrame(metricas_modelos)

print("\n\nTABLA COMPARATIVA")
print("=" * 70)
print(df_metricas.to_string(index=False))
```

```
MÉTRICAS DE EVALUACIÓN - CONJUNTO DE PRUEBA
======================================================================

OLS
--------------------------------------------------
  MSE:          1.0902
```

```
  RMSE:          1.0441
  MAE:           0.8652
  R²:            0.8480
  R² Ajustado:   0.8427

Ridge
--------------------------------------------------
  MSE:           1.0902
  RMSE:          1.0441
  MAE:           0.8652
  R²:            0.8480
  R² Ajustado:   0.8427

Lasso
--------------------------------------------------
  MSE:           1.0907
  RMSE:          1.0443
  MAE:           0.8654
  R²:            0.8479
  R² Ajustado:   0.8426

KNN
--------------------------------------------------
  MSE:           1.5612
  RMSE:          1.2495
  MAE:           0.9792
  R²:            0.7823
  R² Ajustado:   0.7747


TABLA COMPARATIVA
======================================================================
modelo      mse     rmse      mae       r2   r2_adj
   OLS 1.090161 1.044108 0.865203 0.848011 0.842678
 Ridge 1.090163 1.044109 0.865204 0.848011 0.842678
 Lasso 1.090653 1.044343 0.865375 0.847943 0.842607
   KNN 1.561182 1.249473 0.979177 0.782342 0.774705
```

9 Comparación de modelos

9.1 Tabla y visualización comparativa

```
# Visualización comparativa
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# R²
axes[0, 0].bar(df_metricas['modelo'], df_metricas['r2'], alpha=0.7)
axes[0, 0].set_ylabel('R²')
```

```
axes[0, 0].set_title('R² por Modelo')
axes[0, 0].set_ylim([0, 1])
axes[0, 0].grid(True, alpha=0.3, axis='y')

# RMSE
axes[0, 1].bar(df_metricas['modelo'], df_metricas['rmse'],
               alpha=0.7, color='orange')
axes[0, 1].set_ylabel('RMSE')
axes[0, 1].set_title('RMSE por Modelo (menor es mejor)')
axes[0, 1].grid(True, alpha=0.3, axis='y')

# MAE
axes[1, 0].bar(df_metricas['modelo'], df_metricas['mae'],
               alpha=0.7, color='green')
axes[1, 0].set_ylabel('MAE')
axes[1, 0].set_title('MAE por Modelo (menor es mejor)')
axes[1, 0].grid(True, alpha=0.3, axis='y')

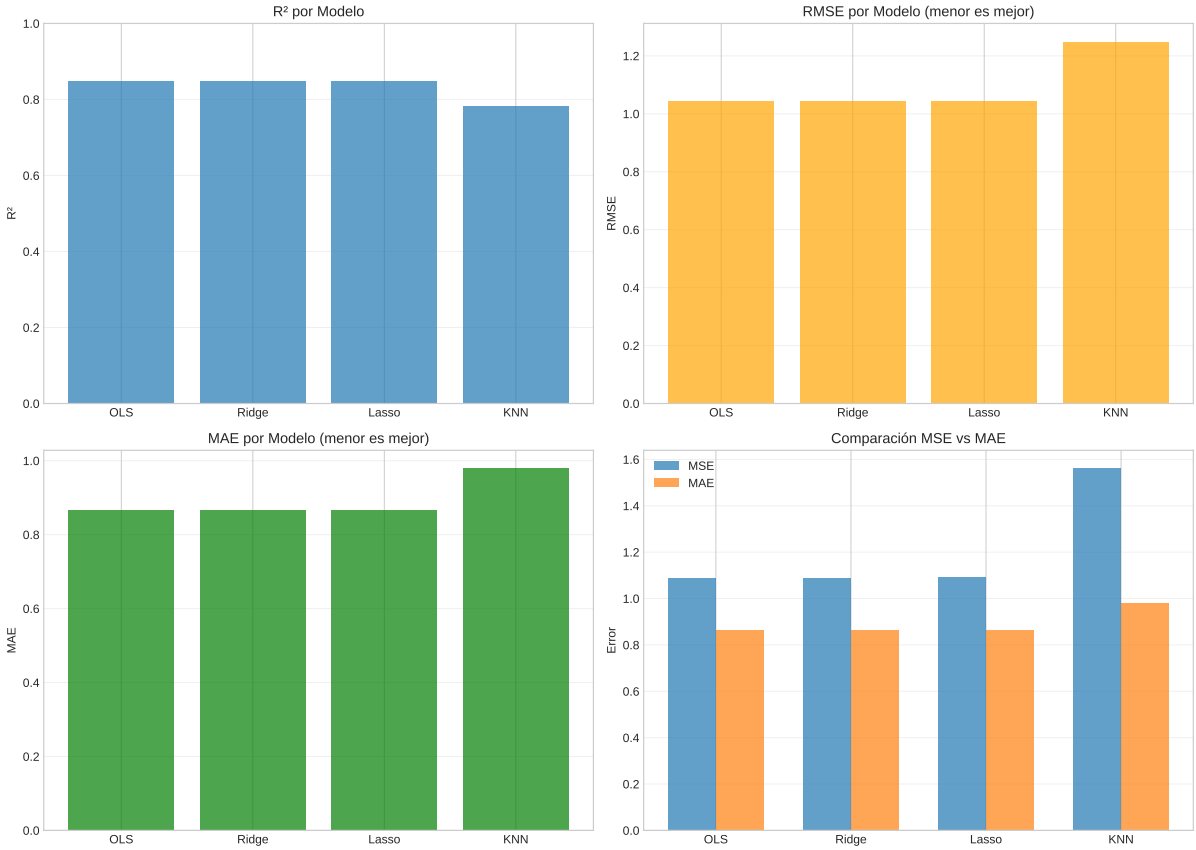
# Comparación múltiple
x = np.arange(len(df_metricas))
width = 0.35

axes[1, 1].bar(x - width/2, df_metricas['mse'], width,
               label='MSE', alpha=0.7)
axes[1, 1].bar(x + width/2, df_metricas['mae'], width,
               label='MAE', alpha=0.7)
axes[1, 1].set_xticks(x)
axes[1, 1].set_xticklabels(df_metricas['modelo'])
axes[1, 1].set_ylabel('Error')
axes[1, 1].set_title('Comparación MSE vs MAE')
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.savefig('model_comparison.png', dpi=300, bbox_inches='tight')
print("\nComparación guardada como 'model_comparison.png'")

# Guardar tabla
df_metricas.to_csv('metricas_comparacion.csv', index=False)
print("Tabla guardada como 'metricas_comparacion.csv'")
```

```
Comparación guardada como 'model_comparison.png'
Tabla guardada como 'metricas_comparacion.csv'
```



10 Aplicaciones económicas

10.1 Aplicación 1: Predicción de precios de vivienda

```
print("\n\n\nAPLICACIÓN: PREDICCIÓN DE PRECIOS DE VIVIENDA")
print("=" * 70)

# Generar datos sintéticos de viviendas
np.random.seed(42)
n_casas = 500

datos_vivienda = pd.DataFrame({
    'area_m2': np.random.uniform(50, 300, n_casas),
    'num_habitaciones': np.random.randint(1, 6, n_casas),
    'num_banos': np.random.randint(1, 4, n_casas),
    'antiguedad_anos': np.random.uniform(0, 50, n_casas),
    'distancia_centro_km': np.random.uniform(0.5, 30, n_casas),
    'area_jardin_m2': np.random.uniform(0, 200, n_casas),
    'tiene_cochera': np.random.choice([0, 1], n_casas, p=[0.3, 0.7]),
    'piso': np.random.randint(1, 15, n_casas)
})

# Generar precio (en miles de soles)
precio = (
    datos_vivienda['area_m2'] * 5 +
```

```
    datos_vivienda['num_habitaciones'] * 50 +
    datos_vivienda['num_banos'] * 30 +
    -datos_vivienda['antiguedad_anos'] * 2 +
    -datos_vivienda['distancia_centro_km'] * 8 +
    datos_vivienda['area_jardin_m2'] * 1.5 +
    datos_vivienda['tiene_cochera'] * 100 +
    np.random.normal(0, 50, n_casas)
)

datos_vivienda['precio_miles'] = precio

print(f"\nDataset de viviendas:")
print(f"  Total viviendas: {n_casas}")
print(f"\nEstadísticas de precio (miles de soles):")
print(datos_vivienda['precio_miles'].describe())

# Preparar datos
X_vivienda = datos_vivienda.drop('precio_miles', axis=1)
y_vivienda = datos_vivienda['precio_miles']

X_train_v, X_test_v, y_train_v, y_test_v = train_test_split(
    X_vivienda, y_vivienda, test_size=0.2, random_state=42
)

# Estandarizar
scaler_v = StandardScaler()
X_train_v_scaled = scaler_v.fit_transform(X_train_v)
X_test_v_scaled = scaler_v.transform(X_test_v)

# Entrenar múltiples modelos
print("\n\nRESULTADOS DE MODELOS")
print("=" * 70)

modelos_vivienda = {
    'OLS': LinearRegression(),
    'Ridge': Ridge(alpha=10),
    'Lasso': Lasso(alpha=1, max_iter=10000),
    'Random Forest': RandomForestRegressor(n_estimators=100, random_state=42)
}

resultados_vivienda = []

for nombre, modelo in modelos_vivienda.items():
    # Entrenar (algunos modelos necesitan datos escalados)
    if nombre in ['OLS', 'Ridge', 'Lasso']:
        modelo.fit(X_train_v_scaled, y_train_v)
        y_pred = modelo.predict(X_test_v_scaled)
```

```
    else:
        modelo.fit(X_train_v, y_train_v)
        y_pred = modelo.predict(X_test_v)

    # Métricas
    r2 = r2_score(y_test_v, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test_v, y_pred))
    mae = mean_absolute_error(y_test_v, y_pred)

    resultados_vivienda.append({
        'Modelo': nombre,
        'R²': r2,
        'RMSE': rmse,
        'MAE': mae
    })

    print(f"\n{nombre}:")
    print(f"  R²:   {r2:.4f}")
    print(f"  RMSE: {rmse:.2f} miles de soles")
    print(f"  MAE:  {mae:.2f} miles de soles")

# Importancia de características (Random Forest)
print(f"\n\nIMPORTANCIA DE CARACTERÍSTICAS (Random Forest)")
print("=" * 70)

importancias_vivienda = pd.DataFrame({
    'caracteristica': X_vivienda.columns,
    'importancia': modelos_vivienda['Random Forest'].feature_importances_
}).sort_values('importancia', ascending=False)

for _, row in importancias_vivienda.iterrows():
    barra = ' ' * int(row['importancia'] * 100)
    print(f"  {row['caracteristica']:<25} {row['importancia']:.4f} {barra}")

# Ejemplo de predicción
print(f"\n\nEJEMPLO DE PREDICCIÓN")
print("=" * 70)

casa_ejemplo = pd.DataFrame({
    'area_m2': [150],
    'num_habitaciones': [3],
    'num_banos': [2],
    'antiguedad_anos': [10],
    'distancia_centro_km': [5],
    'area_jardin_m2': [50],
    'tiene_cochera': [1],
    'piso': [3]
```

```
})

print("\nCaracterísticas de la vivienda:")
for col in casa_ejemplo.columns:
    print(f"  {col}: {casa_ejemplo[col].values[0]}")

# Predecir con cada modelo
print(f"\nPredicciones de precio (miles de soles):")
casa_ejemplo_scaled = scaler_v.transform(casa_ejemplo)

for nombre, modelo in modelos_vivienda.items():
    if nombre in ['OLS', 'Ridge', 'Lasso']:
        pred = modelo.predict(casa_ejemplo_scaled)[0]
    else:
        pred = modelo.predict(casa_ejemplo)[0]

    print(f"  {nombre:<15} S/ {pred:,.0f} mil")
```

```
APLICACIÓN: PREDICCIÓN DE PRECIOS DE VIVIENDA
======================================================================

Dataset de viviendas:
  Total viviendas: 500

Estadísticas de precio (miles de soles):
count     500.000000
mean     1137.639883
std       394.316529
min       273.313950
25%       806.868893
50%      1130.556820
75%      1453.350005
max      2089.835315
Name: precio_miles, dtype: float64


RESULTADOS DE MODELOS
======================================================================

OLS:
  R²:   0.9788
  RMSE: 54.77 miles de soles
  MAE:  42.88 miles de soles

Ridge:
  R²:   0.9794
```

```
  RMSE: 54.01 miles de soles
  MAE:  43.23 miles de soles

Lasso:
  R²:   0.9795
  RMSE: 53.81 miles de soles
  MAE:  42.35 miles de soles

Random Forest:
  R²:   0.9544
  RMSE: 80.31 miles de soles
  MAE:  67.31 miles de soles


IMPORTANCIA DE CARACTERÍSTICAS (Random Forest)
======================================================================
  area_m2                   0.8669
  area_jardin_m2            0.0562
  distancia_centro_km       0.0348
  num_habitaciones          0.0192
  antiguedad_anos           0.0105
  piso                      0.0054
  tiene_cochera             0.0039
  num_banos                 0.0030


EJEMPLO DE PREDICCIÓN
======================================================================

Características de la vivienda:
  area_m2: 150
  num_habitaciones: 3
  num_banos: 2
  antiguedad_anos: 10
  distancia_centro_km: 5
  area_jardin_m2: 50
  tiene_cochera: 1
  piso: 3

Predicciones de precio (miles de soles):
  OLS             S/ 1,088 mil
  Ridge           S/ 1,089 mil
  Lasso           S/ 1,085 mil
  Random Forest   S/ 1,063 mil
```

10.2 Aplicación 2: Predicción de ventas

```
print("\n\n\nAPLICACIÓN: PREDICCIÓN DE VENTAS")
print("=" * 70)

# Generar datos de ventas
np.random.seed(42)
n_productos = 400

datos_ventas = pd.DataFrame({
    'precio': np.random.uniform(10, 200, n_productos),
    'inversion_publicidad': np.random.uniform(0, 10000, n_productos),
    'num_competidores': np.random.randint(1, 10, n_productos),
    'calificacion_producto': np.random.uniform(2, 5, n_productos),
    'inventario_disponible': np.random.randint(10, 1000, n_productos),
    'descuento_pct': np.random.uniform(0, 30, n_productos),
    'temporada_alta': np.random.choice([0, 1], n_productos, p=[0.7, 0.3])
})

# Generar ventas con relación no lineal
ventas = (
    -0.5 * datos_ventas['precio'] +
    0.8 * np.sqrt(datos_ventas['inversion_publicidad']) +
    -10 * datos_ventas['num_competidores'] +
    100 * datos_ventas['calificacion_producto'] +
    0.05 * datos_ventas['inventario_disponible'] +
    3 * datos_ventas['descuento_pct'] +
    150 * datos_ventas['temporada_alta'] +
    np.random.normal(0, 50, n_productos)
)

datos_ventas['ventas_mensuales'] = np.maximum(ventas, 0)  # No ventas negativas

print(f"\nDataset de ventas:")
print(f"  Total productos: {n_productos}")
print(f"\nEstadísticas de ventas mensuales:")
print(datos_ventas['ventas_mensuales'].describe())

# Preparar datos
X_ventas = datos_ventas.drop('ventas_mensuales', axis=1)
y_ventas = datos_ventas['ventas_mensuales']

X_train_s, X_test_s, y_train_s, y_test_s = train_test_split(
    X_ventas, y_ventas, test_size=0.2, random_state=42
)

# Probar regresión polinomial
```

```
print("\n\nREGRESIÓN POLINOMIAL PARA VENTAS")
print("=" * 70)

# Crear características polinomiales de grado 2
poly = PolynomialFeatures(degree=2, include_bias=False)
X_train_s_poly = poly.fit_transform(X_train_s)
X_test_s_poly = poly.transform(X_test_s)

# Entrenar modelos
modelos_ventas = {
    'Lineal': LinearRegression(),
    'Polinomial (grado 2)': LinearRegression(),
    'Ridge Polinomial': Ridge(alpha=100),
    'Random Forest': RandomForestRegressor(n_estimators=200, max_depth=10, random_sta
}

# Lineal
modelos_ventas['Lineal'].fit(X_train_s, y_train_s)
y_pred_lineal = modelos_ventas['Lineal'].predict(X_test_s)

# Polinomial
modelos_ventas['Polinomial (grado 2)'].fit(X_train_s_poly, y_train_s)
y_pred_poly = modelos_ventas['Polinomial (grado 2)'].predict(X_test_s_poly)

# Ridge Polinomial
modelos_ventas['Ridge Polinomial'].fit(X_train_s_poly, y_train_s)
y_pred_ridge_poly = modelos_ventas['Ridge Polinomial'].predict(X_test_s_poly)

# Random Forest
modelos_ventas['Random Forest'].fit(X_train_s, y_train_s)
y_pred_rf = modelos_ventas['Random Forest'].predict(X_test_s)

# Evaluar
predicciones = {
    'Lineal': y_pred_lineal,
    'Polinomial (grado 2)': y_pred_poly,
    'Ridge Polinomial': y_pred_ridge_poly,
    'Random Forest': y_pred_rf
}

print(f"{'Modelo':<25} {'R²':<10} {'RMSE':<15} {'MAE':<15}")
print("-" * 70)

for nombre, y_pred in predicciones.items():
    r2 = r2_score(y_test_s, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test_s, y_pred))
    mae = mean_absolute_error(y_test_s, y_pred)
```

```
    print(f"{nombre:<25} {r2:<10.4f} {rmse:<15.2f} {mae:<15.2f}")

# Análisis de elasticidad precio
print(f"\n\nANÁLISIS DE ELASTICIDAD PRECIO")
print("=" * 70)

# Usar modelo Random Forest para análisis
precios_test = np.linspace(10, 200, 50)
datos_base = X_ventas.mean().to_dict()

ventas_por_precio = []

for precio in precios_test:
    datos_pred = pd.DataFrame([datos_base])
    datos_pred['precio'] = precio
    ventas_pred = modelos_ventas['Random Forest'].predict(datos_pred)[0]
    ventas_por_precio.append(ventas_pred)

# Graficar curva de demanda
plt.figure(figsize=(10, 6))
plt.plot(precios_test, ventas_por_precio, linewidth=2)
plt.xlabel('Precio')
plt.ylabel('Ventas mensuales predichas')
plt.title('Curva de Demanda Estimada')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('curva_demanda.png', dpi=300, bbox_inches='tight')
print("\nCurva de demanda guardada como 'curva_demanda.png'")

# Calcular elasticidad en diferentes puntos
idx_medio = len(precios_test) // 2
precio_medio = precios_test[idx_medio]
ventas_medio = ventas_por_precio[idx_medio]

# Elasticidad = (ΔQ/Q) / (ΔP/P)
delta_p = precios_test[idx_medio + 1] - precios_test[idx_medio]
delta_q = ventas_por_precio[idx_medio + 1] - ventas_por_precio[idx_medio]

elasticidad = (delta_q / ventas_medio) / (delta_p / precio_medio)

print(f"\nElasticidad precio en punto medio:")
print(f"  Precio: S/ {precio_medio:.2f}")
print(f"  Elasticidad: {elasticidad:.3f}")

if abs(elasticidad) > 1:
    print(f"  → Demanda elástica (sensible al precio)")
else:
```

```
    print(f"  → Demanda inelástica (poco sensible al precio)")
```

```
APLICACIÓN: PREDICCIÓN DE VENTAS
======================================================================

Dataset de ventas:
  Total productos: 400

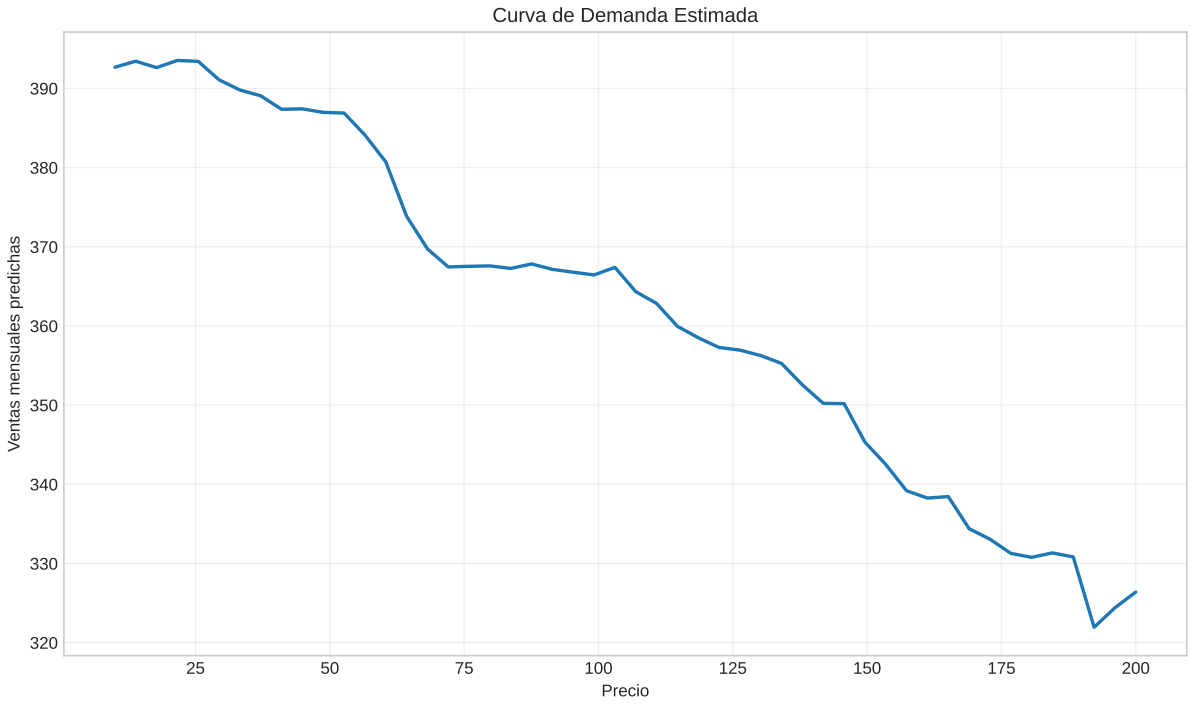
Estadísticas de ventas mensuales:
count    400.000000
mean     413.779329
std      133.711254
min       83.286944
25%      316.421096
50%      412.196200
75%      514.370629
max      799.735141
Name: ventas_mensuales, dtype: float64


REGRESIÓN POLINOMIAL PARA VENTAS
======================================================================
Modelo                    R²         RMSE            MAE
----------------------------------------------------------------------
Lineal                    0.8470     50.32           40.20
Polinomial (grado 2)      0.8329     52.58           43.27
Ridge Polinomial          0.8417     51.18           42.40
Random Forest             0.7766     60.80           49.69


ANÁLISIS DE ELASTICIDAD PRECIO
======================================================================

Curva de demanda guardada como 'curva_demanda.png'

Elasticidad precio en punto medio:
  Precio: S/ 106.94
  Elasticidad: -0.113
  → Demanda inelástica (poco sensible al precio)
```



11 Ejercicios prácticos

11.1 Ejercicio: Validación cruzada

```
print("\n\n\nEJERCICIO: VALIDACIÓN CRUZADA")
print("=" * 70)

# Validación cruzada con diferentes modelos
from sklearn.model_selection import cross_validate

modelos_cv = {
    'OLS': LinearRegression(),
    'Ridge ( =1)': Ridge(alpha=1),
    'Ridge ( =10)': Ridge(alpha=10),
    'Lasso ( =0.1)': Lasso(alpha=0.1, max_iter=10000),
    'Lasso ( =1)': Lasso(alpha=1, max_iter=10000)
}

print("\nValidación cruzada (5-fold):")
print(f"{'Modelo':<20} {'R² medio':<12} {'R² std':<12} {'RMSE medio':<15}")
print("-" * 70)

resultados_cv = []

for nombre, modelo in modelos_cv.items():
    # Validación cruzada
    cv_results = cross_validate(
        modelo, X, Y,
```

```
        cv=5,
        scoring=['r2', 'neg_mean_squared_error'],
        return_train_score=False
    )

    r2_medio = cv_results['test_r2'].mean()
    r2_std = cv_results['test_r2'].std()
    mse_medio = -cv_results['test_neg_mean_squared_error'].mean()
    rmse_medio = np.sqrt(mse_medio)

    resultados_cv.append({
        'modelo': nombre,
        'r2_medio': r2_medio,
        'r2_std': r2_std,
        'rmse_medio': rmse_medio
    })

    print(f"{nombre:<20} {r2_medio:<12.4f} {r2_std:<12.4f} {rmse_medio:<15.4f}")

# Mejor modelo por CV
df_cv = pd.DataFrame(resultados_cv)
mejor_modelo_cv = df_cv.loc[df_cv['r2_medio'].idxmax(), 'modelo']

print(f"\n\nMejor modelo según validación cruzada: {mejor_modelo_cv}")
print("(Balance entre performance y generalización)")
```

```
EJERCICIO: VALIDACIÓN CRUZADA
======================================================================

Validación cruzada (5-fold):
Modelo               R² medio     R² std       RMSE medio
----------------------------------------------------------------------
OLS                  0.8546       0.0204       1.0002
Ridge ( =1)          0.8547       0.0208       0.9999
Ridge ( =10)         0.8524       0.0247       1.0091
Lasso ( =0.1)        0.8525       0.0247       1.0087
Lasso ( =1)          0.5366       0.0332       1.7877


Mejor modelo según validación cruzada: Ridge ( =1)
(Balance entre performance y generalización)
```

12 Conclusión

En esta guía hemos explorado los métodos de regresión:
Mínimos Cuadrados Ordinarios (OLS)

Método base, interpretable, con solución analítica. Sensible a outliers y multicolinealidad.

Regresión Ridge
Penalización L2. Reduce magnitud de coeficientes. Útil con variables correlacionadas.
Regresión Lasso
Penalización L1. Selección automática de variables. Produce modelos sparse.
Elastic Net
Combina Ridge y Lasso. Balance entre selección y estabilidad.
Regresión Polinomial
Captura relaciones no lineales. Propenso a overfitting con grados altos.
K-Nearest Neighbors
No paramétrico. Captura patrones complejos. Sensible a escala y lento.
Métricas: R^2, MSE, RMSE, MAE
Diferentes perspectivas del error. R^2 para varianza explicada. RMSE/MAE para magnitud del error.

13 Próximos pasos

En la siguiente guía (Guía 11: Validación Cruzada y Composición del Modelo) exploraremos:

- Validación cruzada (k-fold, stratified)
- Grid search para hiperparámetros
- Curvas de aprendizaje
- Contribución de features
- Bias-variance tradeoff

14 Recursos adicionales

Para profundizar en regresión:

- Scikit-learn Regression: scikit-learn.org/stable/supervised_learning.html#supervised-learning
- Introduction to Statistical Learning (ISLR) - Capítulos 3, 6
- Elements of Statistical Learning (ESL)
- Applied Econometrics with Python

15 Publicaciones Similares

Si te interesó este artículo, te recomendamos que explores otros blogs y recursos relacionados que pueden ampliar tus conocimientos. Aquí te dejo algunas sugerencias:

1. Instalacion De Anaconda
2. Configurar Entorno Virtual Python Anaconda
3. 01 Introducion A La Programacion Con Python
4. 02 Variables Expresiones Y Statements Con Python
5. 03 Objetos De Python
6. 04 Ejecucion Condicional Con Python
7. 05 Iteraciones Con Python
8. 06 Funciones Con Python
9. 07 Dataframes Con Python

10. 08 Prediccion Y Metrica De Performance Con Python
11. 09 Metodos De Machine Learning Para Clasificacion Con Python
12. 10 Metodos De Machine Learning Para Regresion Con Python
13. 11 Validacion Cruzada Y Composicion Del Modelo Con Python
14. Visualizacion De Datos Con Python

Esperamos que encuentres estas publicaciones igualmente interesantes y útiles. ¡Disfruta de la lectura!